

Effiziente Subgraph-Suche in Datenbanken biologischer Netzwerke: Realisierung und Analyse von Gen-DB

Stephan Epp

23. Januar 2026

Inhaltsverzeichnis

1	Einführung	3
1.1	Beitrag dieser Arbeit	3
1.2	Aufbau der Arbeit	3
2	Grundlagen	4
2.1	Graphen und Adjazenzmatrizen	4
2.2	Signaturen und Subgraph-Relation	4
2.3	Der Subgraph Algorithmus	4
3	Systemarchitektur von Gen-DB	5
3.1	Komponentenübersicht	5
3.2	Datenmodell	5
3.3	UML-Architekturmodelle von Gen-DB	6
4	Such-Pipeline: Formalisierung und Beweise	7
4.1	Problemformulierung	7
4.2	SQL-Vorauswahl	9
4.3	SQL-Select-Statement	10
4.4	Gesamtkomplexität der Pipeline	10
4.5	Korrektheit der Gesamtpipeline	10
4.6	Effiziente Suche	11
5	Empirische Evaluation	12
5.1	Versuchsaufbau	12
5.2	Generierungsstatistik	13
5.3	Visualisierung 1: Knoten- und Kantenverteilung	13
5.4	Visualisierung 2: Pre-Filter-Effektivität	13
5.5	Visualisierung 3: Skalierungsverhalten der Suchzeit	14
5.6	Visualisierung 4: Trefferverteilung nach Match-Typ	14
5.7	Diskussion	14

6	Experimenteller Vergleich	15
6.1	Versuchsanordnung	15
6.2	Ergebnis 1: Laufzeitvergleich bei festem $n_A = 5$, wachsendem n_B	17
6.3	Der Subgraph Algorithmus für $n_B \geq 14$	18
6.4	Einordnung in die Literatur effizienter Subgraph-Isomorphie-Verfahren . . .	21
7	Schluss	23
7.1	Indexierung über Signatur-Hashes	23
7.2	Mehrdimensionale Indexstrukturen (R-Baum über (n_k, m_k))	24
7.3	Parallelisierung der Subgraph-Vergleiche	24
7.4	GPU-Beschleunigung der Signatur- und LCS-Berechnung	24
7.5	Approximative Suche und geschichtete Stichproben	25
7.6	Erweiterung auf gewichtete und beschriftete Subgraph-Isomorphie	25
7.7	Inkrementelle Aktualisierung der Signatur-Indizes	25
7.8	Caching häufiger Queries	26
7.9	Visuelle Exploration und Graph-Layout	26
7.10	Benchmark-Suite und kontinuierliche Performanzmessung	26
7.11	Hybridstrategie: Adaptive Verfahrenswahl nach Kandidatengröße	26
7.12	Erweiterung des Vergleichsbenchmarks	27
7.13	Formale Charakterisierung des Übergangsbereichs θ	28
7.14	Priorisierung der nächsten Entwicklungsschritte	29
7.15	Zusammenfassung	29

1 Einführung

Biologische Datenbanken enthalten heute Millionen von Netzwerken: metabolische Pfade, Protein-Interaktionsnetzwerke und Gen-Regulationsnetzwerke. Eine zentrale Aufgabe bei der Analyse solcher Datenbanken ist die *Subgraph-Suche*: Gegeben sei ein Query-Netzwerk G (z. B. ein bekanntes Motiv wie ein Teilstück der Glykolyse), gesucht sind alle in der Datenbank gespeicherten Netzwerke G' , die G als Subgraphen enthalten.

Diese Arbeit beschreibt die konkrete Realisierung einer solchen Suche im System *Gen-DB*, einer Full-Stack-Anwendung bestehend aus einem PostgreSQL-Backend, einer FastAPI-REST-API und einem Web-Frontend. Das System nutzt den *Subgraph Algorithmus* [1] mit Laufzeit $O(n^3)$ zur strukturellen Analyse biologischer Netzwerke, dargestellt als Adjazenzmatrizen.

1.1 Beitrag dieser Arbeit

- Formale Beschreibung der Systemarchitektur von Gen-DB und Einbettung des Subgraph Algorithmus in eine relationale Datenbank.
- Formaler Nachweis der Korrektheit und Komplexität des Such-Pipeline-Prozesses (Pre-Filtering + Subgraph-Vergleich).
- Empirische Evaluation an einer synthetisch generierten Datenbank mit 1.000.000 biologischen Netzwerken.
- Veranschaulichung der Ergebnisse durch matplotlib-Diagramme: Laufzeitverteilung, Pre-Filter-Effektivität, Skalierungsverhalten und Trefferverteilung.
- Ausführlicher Ausblick auf Erweiterungen (Indexierung, Parallelisierung, approximative Suche, GPU-Beschleunigung).

1.2 Aufbau der Arbeit

Zunächst führt Abschnitt 2 in biologische Netzwerke, ihre Adjazenzmatrix-Repräsentation und den Subgraph Algorithmus ein. Abschnitt 3 beschreibt die Systemarchitektur und das Datenmodell von Gen-DB und erläutert anhand von UML-Paket- und Klassendiagrammen den Aufbau der Anwendung. Abschnitt 4 formalisiert die Such-Pipeline, insbesondere die Vorauswahl anhand von Knoten- und Kantenanzahl, und untersucht deren Korrektheit und Laufzeitkomplexität; ein Serverprotokoll veranschaulicht dort zusätzlich den Ablauf einer konkreten Suche. Abschnitt 5 stellt den Versuchsaufbau sowie die Ergebnisse zur generierten Datenbank und Suche vor und schließt mit einer Diskussion. Abschnitt 6 widmet sich dem experimentellen Vergleich mit etablierten Verfahren, analysiert die beobachteten Laufzeitunterschiede und ordnet die Verfahren in die Fachliteratur ein. Das abschließende Kapitel „Schluss“ (Abschnitt 7) bündelt den Ausblick mit möglichen Erweiterungen und offenen Forschungsfragen; seine letzte Untersektion fasst die wesentlichen Ergebnisse zusammen.

2 Grundlagen

2.1 Graphen und Adjazenzmatrizen

Definition 1 (Biologisches Netzwerk). Ein biologisches Netzwerk ist ein gerichteter Graph $G = (V, E)$ mit Knotenmenge $V = \{v_1, \dots, v_n\}$ (Metabolite, Proteine oder Gene) und Kantenmenge $E \subseteq V \times V$ (Reaktionen, Interaktionen oder Regulationsbeziehungen). Jedem Knoten v_i ist ein Label $\ell(v_i) \in \Sigma_{\text{Bio}}$ aus einem biologischen Vokabular zugeordnet.

Definition 2 (Adjazenzmatrix-Repräsentation). Ein Netzwerk G mit n Knoten wird in Gen-DB als Tupel

$$G = (L, A) \quad \text{mit } L \in \Sigma_{\text{Bio}}^n, A \in \{0, 1\}^{n \times n}$$

gespeichert, wobei L der Vektor der Knotenlabels und A die Adjazenzmatrix mit $A_{ij} = 1 \Leftrightarrow (v_i, v_j) \in E$ ist.

2.2 Signaturen und Subgraph-Relation

Definition 3 (Spaltensignatur). Für eine Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ ist die Signatur der Spalte j definiert als

$$\sigma_j(A) = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Definition 4 (Signatur-Array und Hash). Das Signatur-Array von A ist $\Sigma(A) = (\sigma_0(A), \dots, \sigma_{n-1}(A))$. Der Signatur-Hash ist

$$h(A) = \text{SHA256}(\text{str}(\Sigma(A))).$$

Lemma 1 (Injektivität der Signaturfunktion). Die Abbildung $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$, $(c, j) \mapsto \sum_{i=0}^{n-1} c_i 2^i + j 2^n$ ist injektiv.

Beweis. Sei $\sigma(c, j) = \sigma(c', j')$. Da $\sum_{i=0}^{n-1} c_i 2^i \in [0, 2^n - 1]$ und $j 2^n, j' 2^n$ Vielfache von 2^n sind, folgt aus der Eindeutigkeit der Division mit Rest (Divisionsalgorithmus für 2^n), dass $j = j'$ und somit $\sum c_i 2^i = \sum c'_i 2^i$. Da die Binärdarstellung eindeutig ist, folgt $c = c'$. $\square$

Definition 5 (Subgraph-Relation, zyklisch). Für Adjazenzmatrizen $A \in \{0, 1\}^{n_A \times n_A}$ und $B \in \{0, 1\}^{n_B \times n_B}$ mit $n_A \leq n_B$ schreiben wir $A \preceq B$ (“ A ist zyklischer Subgraph von B ”), wenn es eine zyklische Rotation $r \in \{0, \dots, n_B - 1\}$ und eine Startposition s gibt, sodass die Zeilenkomponenten der Signaturen von A als längste gemeinsame Teilsequenz (LCS) der Länge ≥ 2 in der rotierten Zeilenkomponentensequenz von B ab Position s auftreten.

2.3 Der Subgraph Algorithmus

Der Subgraph Algorithmus [1] berechnet für zwei Matrizen A, B in drei Schritten, ob $A \preceq B$, $B \preceq A$, beide äquivalent oder unvergleichbar sind:

1. Berechne $\Sigma(A)$ und $\Sigma(B)$ — Aufwand $O(n_A^2 + n_B^2)$.
2. Extrahiere Zeilenkomponenten $\rho(A) = (\sigma_j(A) \bmod 2^{n_A})_j$, analog für B .
3. Für jede der n_B zyklischen Rotationen von $\rho(B)$: Vergleiche mit $\rho(A)$ via LCS in $O(n_A \cdot n_B)$.

Satz 1 (Laufzeit des Subgraph Algorithmus). Für $n = \max(n_A, n_B)$ benötigt der Subgraph Algorithmus $O(n^3)$ Zeit.

Beweis. Schritt 1 ist $O(n^2)$. Schritt 3 besteht aus $n_B \leq n$ Rotationen, jede mit LCS-Vergleich in $O(n_A n_B) \subseteq O(n^2)$, insgesamt $O(n^3)$. Schritt 2 ist $O(n)$. Die Summe ist $O(n^2) + O(n) + O(n^3) = O(n^3)$. $\square$

Eine vollständige Optimalitätsanalyse $\Omega(n^{3-\epsilon})$ unter SETH findet sich in [1]; sie wird hier als gegeben vorausgesetzt und in Abschnitt 4 auf den Datenbankkontext übertragen.

3 Systemarchitektur von Gen-DB

3.1 Komponentenübersicht

Gen-DB besteht aus drei Schichten:

- **Frontend** (HTML/CSS/JavaScript): Erstellung, Anzeige und Suche von Netzwerken über REST-Aufrufe.
- **Backend** (FastAPI, Python): REST-Endpunkte `/api/networks`, `/api/networks/{id}`, `/api/networks/search`; Modul `crud.py` implementiert Signaturberechnung und ruft den Subgraph Algorithmus auf.
- **Datenbank** (PostgreSQL): Tabellen `biological_networks` (Metadaten: Name, Typ, Organismus, Knoten-/Kantenzahl) und `network_matrices` (Knotenlabels, Adjazenzmatrix, Signatur-Array, Signatur-Hash).

Definition 6 (Datenbankzustand). Der Datenbankzustand zum Zeitpunkt t ist eine endliche Menge

$$\mathcal{D}_t = \{(id_k, L_k, A_k, \Sigma_k, h_k) : k = 1, \dots, N_t\},$$

wobei $N_t = |\mathcal{D}_t|$ die Anzahl gespeicherter Netzwerke ist. In der vorliegenden Evaluation gilt $N_t \approx 1.000.002$.

3.2 Datenmodell

Die Speicherung erfolgt normalisiert über zwei 1:1-verknüpfte Relationen:

`biological_networks(network__id, name, type, organism, node__count, edge__count)`

`network_matrices(network__id, L, A,  $\Sigma$ , h)`

Die Spalten `node__count` und `edge__count` entsprechen $n_k = |V_k|$ bzw. $m_k = |E_k| = \sum_{i,j} (A_k)_{ij}$ und werden für das Pre-Filtering (Abschnitt 4.2) materialisiert, um eine vollständige Tabellenabtastung zu vermeiden.

3.3 UML-Architekturmodelle von Gen-DB

Die beiden UML-Diagramme in den Abbildungen 1 und 2 dokumentieren die Struktur von Gen-DB auf Paket- und Klassenebene und bilden damit die konzeptionelle Grundlage für die in den folgenden Abschnitten formalisierten Such- und Datenbankprozesse.

Das Paketdiagramm zeigt die fachliche und technische Zerlegung der Anwendung in drei logisch getrennte Bausteine: das Web-Frontend, das Backend mit den API- und Suchkomponenten und die persistente Datenspeicherung in PostgreSQL. Dieser Aufbau ist keine rein dokumentarische Anordnung, sondern entspricht einem klaren Entwurfsmuster für skalierbare Datenbankanwendungen: Die Präsentationsschicht übernimmt keine Datenlogik, sondern stellt lediglich die Interaktion mit dem Benutzer bereit; die Logikschicht kapselt die Kernfunktionalität der Signaturberechnung, des Pre-Filters und des Subgraph-Checks; die Datenbankschicht speichert neben den Netzwerkinhalten auch die maschinenlesbaren Repräsentationen, die für die Suche benötigt werden. Aus Sicht der Systemarchitektur ist deshalb besonders wichtig, dass die Datenflussrichtung zwischen den Schichten eindeutig ist: Ein Nutzer erzeugt oder lädt ein biologisches Netzwerk über das Frontend; die Anfrage wird als API-Request an das Backend weitergeleitet; dort werden Netzwerkdaten interpretiert, normalisiert und in Form von Knotenlabels und Adjazenzmatrizen für den Subgraph-Algorithmus aufbereitet; danach werden die passenden Kandidaten mit SQL-Filterkriterien aus der Datenbank abgerufen und im Ergebnis an die Präsentationsschicht zurückgegeben. Das Paketdiagramm macht damit sichtbar, dass Gen-DB als System nicht nur aus einer einzelnen Anwendung besteht, sondern aus einer zusammengesetzten, modularen Architektur, in der fachliche Trennung und technische Verantwortlichkeit klar voneinander abgegrenzt sind.

Die in der übrigen Arbeit beschriebene Such-Pipeline baut genau auf dieser Struktur auf. Die Datenbank hält nicht nur die Netzwerkmetadaten wie Name, Organismus und Typ, sondern auch die strukturellen Repräsentationen in einer für schnelle Filterung geeigneten Form. Die Backend-Komponenten sind dafür zuständig, die Suchanfrage in eine Folge von Vorfilter-, Vergleichs- und Ergebnisphasen zu übersetzen. Das Paketdiagramm zeigt somit die organisatorische Ebene des Systems: welche Komponenten miteinander interagieren, welche Verantwortlichkeiten sie übernehmen und wo die fachlichen Grenzen verlaufen. Diese Abgrenzung ist für das Verständnis der späteren Komplexitäts- und Korrektheitsbeweise entscheidend, weil sie präzise zeigt, wo der algorithmische Kern liegt und wo rein relationale oder benutzerorientierte Aufgaben ausgelagert werden.

Das Klassendiagramm ergänzt diese Sicht um den objektorientierten bzw. modellorientierten Entwurf der einzelnen Daten- und Dienstekomponenten. Es macht deutlich, dass die Systemlogik nicht auf lose, unstrukturierte Listen von Werten beruht, sondern auf semantisch geordneten Entitäten: Netzwerkobjekte, Knoten- und Kanteninformationen, Matrixrepräsentationen, Signaturdaten und Suchresultate sind als getrennte Strukturen modelliert. Ein wesentlicher Entwurfspunkt ist die Trennung zwischen der biologischen Netzwerkanalyse und der relationalen Persistenz. Während die Datenbankrelationen die langfristige Speicherung, Indizierung und Abfrage von Netzwerken garantieren, repräsentiert das Klassendiagramm die fachlichen Informationen im Laufzeitkontext der Anwendung. So sind etwa Metadaten wie Typ, Organismus und Knotenzahl in der Klasse bzw. im Datensatz des Netzwerks als formale Eigenschaften modelliert, während die Adjazenzmatrix und die Signatur-Arrays als strukturgebende Merkmale dienen, die für die Subgraph-Relation maßgeblich sind. Dadurch wird die Verbindung zwischen biologischer Semantik und algorithmisch nutzbarer Repräsentation explizit sichtbar.

Darüber hinaus arbeitet das Klassendiagramm die zentralen Abhängigkeiten zwischen

den Suchkomponenten und den Datenstrukturen heraus. Die Signaturberechnung ist nicht isoliert, sondern Teil einer größeren Verarbeitungskette, in der ein Query-Netzwerk mit den gespeicherten Netzwerken verglichen wird, Kandidaten über notwendige Bedingungen vorgefiltert werden und anschließend der Subgraph-Algorithmus entscheidet, ob ein Treffer tatsächlich vorliegt. Diese Abhängigkeiten sind im UML-Modell als Assoziationen und Verantwortlichkeiten nachvollziehbar: Jede Klasse trägt eine bestimmte Rolle in der Suche, aber die fachliche Koordination geschieht über das Zusammenspiel der Komponenten, nicht über eine monolithische Implementierung. Genau dieser modularisierte Aufbau ist es, der die Skalierbarkeit und Analysefähigkeit von Gen-DB erklärt: Die Datenmodellierung unterstützt die Suche, die Suchlogik verwendet die Repräsentation, und die Systemarchitektur koordiniert die Kommunikationspfade zwischen Nutzer, API und Datenbank.

Die beiden Diagramme zusammen leisten also mehr als nur eine Visualisierung des Quellcodes: Sie dokumentieren die Entwurfslogik des Systems als Ganzes. Sie zeigen, dass Gen-DB ein typisches Full-Stack-System ist, dessen Stärke nicht in einer einzelnen technischen Komponente, sondern in der engen Verknüpfung von semantischer Modellierung, datenbankseitiger Indexierung und algorithmischer Subgraph-Erkennung liegt. Gerade diese Architektur ist der Grund, warum die folgenden Abschnitte die Laufzeit, die Korrektheit und die Kandidatenreduktion formal analysieren können: Die Struktur des Systems ist so klar und modular, dass die mathematische Modellierung der Pipeline direkt auf die reale Implementierung zurückgeführt werden kann.

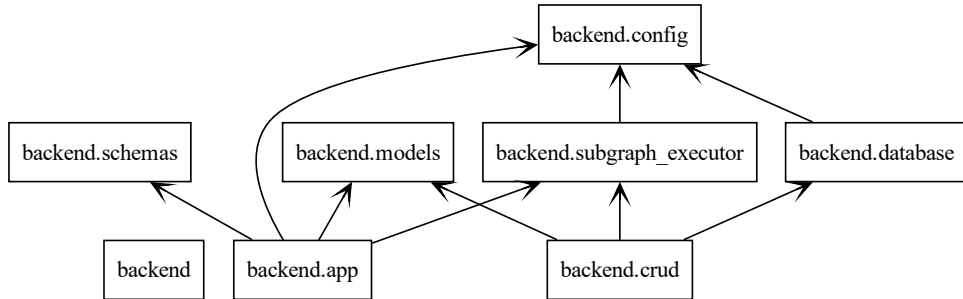


Abbildung 1: UML-Paketdiagramm der Gen-DB-Architektur.

4 Such-Pipeline: Formalisierung und Beweise

4.1 Problemformulierung

Definition 7 (Datenbank-Subgraph-Suche). Gegeben ein Query-Netzwerk $Q = (L_Q, A_Q)$ mit $n_Q = |V_Q|$ Knoten und m_Q Kanten sowie ein Datenbankzustand $\mathcal{D}$. Die Such-Pipeline berechnet die Treffermenge

$$\text{Match}(Q, \mathcal{D}) = \{(id_k, L_k, A_k) \in \mathcal{D} : A_Q \preceq A_k \text{ oder } A_k \preceq A_Q\},$$

annotiert mit dem jeweiligen Treffertyp (**exact** oder **subgraph**).

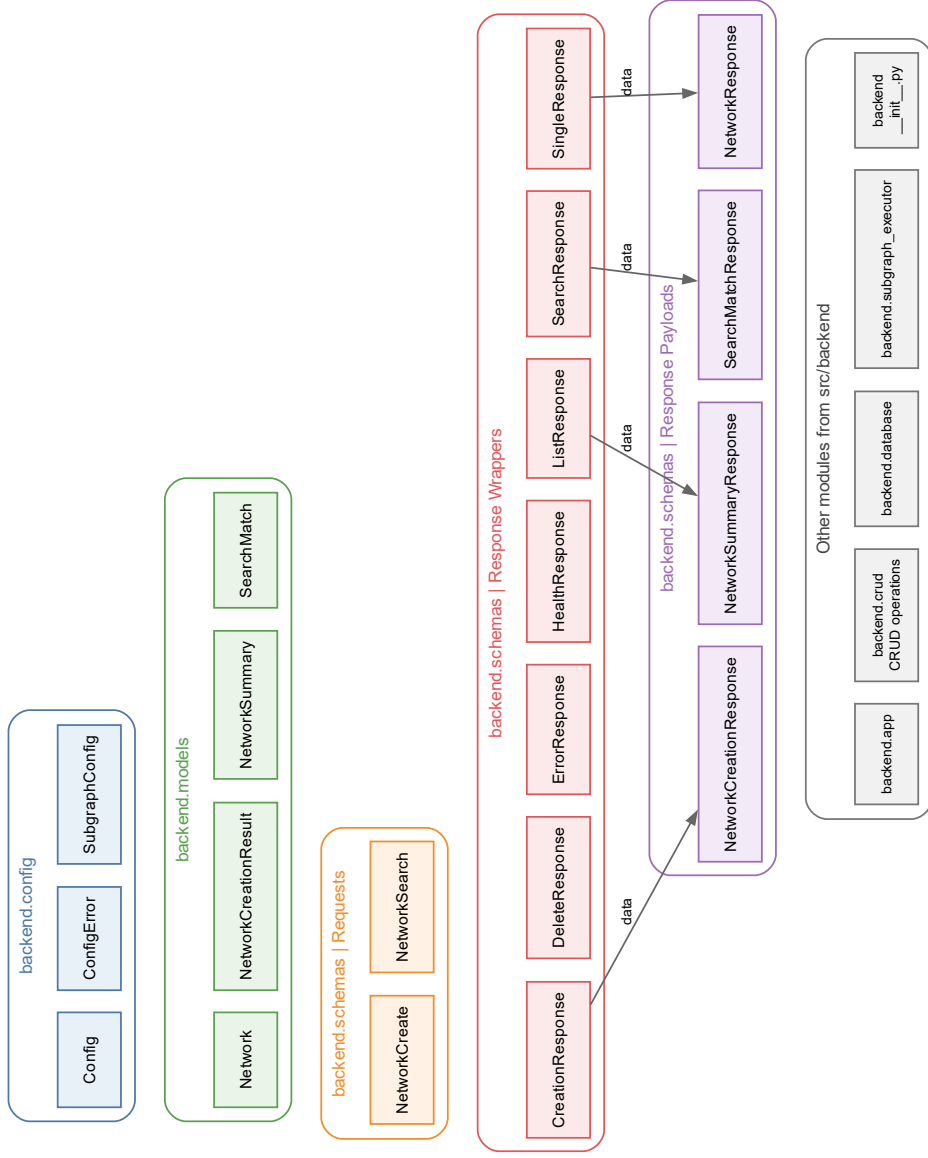


Abbildung 2: UML-Klassendiagramm der Gen-DB-Datenmodellierung und Suche.

4.2 SQL-Vorauswahl

Die zentrale Idee der Gen-DB-Suche besteht darin, den teuren strukturellen Vergleich nicht mit sämtlichen gespeicherten Netzwerken durchzuführen. Stattdessen wird der Suchraum zunächst direkt in der Datenbank durch eine kostengünstige Vorauswahl verkleinert. Dazu werden zwei leicht verfügbare Kennzahlen des Query-Netzwerks mit den entsprechenden Kennzahlen der gespeicherten Netzwerke verglichen: die Knotenzahl und die Kantenanzahl. Für die hier betrachtete Richtung, in der das Query-Netzwerk in einem gespeicherten Netzwerk enthalten sein soll, kann ein gespeichertes Netzwerk mit weniger Knoten oder weniger Kanten kein Treffer sein. Solche Einträge werden daher bereits bei der Selektion ausgeschlossen, bevor ihre Adjazenzmatrix geladen und mit dem Subgraph-Algorithmus untersucht wird.

Die Vorauswahl vergleicht dabei zunächst die Anzahlen, nicht die konkreten Knotenlabels oder die Position jeder einzelnen Kante. Sind beide Schranken erfüllt, bleibt ein Netzwerk ein möglicher Kandidat; ob seine konkrete Struktur tatsächlich zum Query passt, entscheidet erst der anschließende Subgraph-Vergleich. Beispielsweise kann ein Query mit acht Knoten und fünfzehn Kanten nur gespeicherte Netzwerke mit mindestens acht Knoten und mindestens fünfzehn Kanten als Kandidaten zulassen. Ein Eintrag mit sieben Knoten oder vierzehn Kanten lässt sich sicher ausschließen, während ein Eintrag mit acht Knoten und fünfzehn Kanten noch kein Treffer garantiert. Diese Trennung zwischen billiger, datenbankseitiger Kandidatenreduktion und genauer, algorithmischer Prüfung spart insbesondere bei großen Datenbanken unnötige Matrixübertragungen und Subgraph-Vergleiche, ohne für die Richtung $A_Q \preceq A_k$ bei vollständiger Kandidatenbetrachtung echte Treffer zu verlieren. Die folgenden Bedingungen formalisieren diese Vorauswahl.

Lemma 2 (Notwendige Bedingung für $\preceq$). Wenn $A_Q \preceq A_k$, so gilt $n_Q \leq n_k$ und $m_Q \leq m_k$.

Beweis. Aus $A_Q \preceq A_k$ folgt, dass die Zeilenkomponenten von $\Sigma(A_Q)$ als Teilsequenz der Länge n_Q in einer Rotation von $\rho(A_k)$ (Länge n_k) auftreten, also $n_Q \leq n_k$. Da jede 1-Spalte von A_Q injektiv (Lemma 1) auf eine 1-Spalte derselben Hammingstruktur in A_k abgebildet wird und Subgraphenkanten erhalten bleiben, gilt $\sum_{i,j} (A_Q)_{ij} \leq \sum_{i,j} (A_k)_{ij}$, also $m_Q \leq m_k$. $\square$

Korollar 1 (Korrektheit des Pre-Filters). Sei

$$\mathcal{C}(Q, \mathcal{D}) = \{(id_k, L_k, A_k) \in \mathcal{D} : n_k \geq n_Q \wedge m_k \geq m_Q\}.$$

Dann gilt $\text{Match}(Q, \mathcal{D}) \cap \{(id_k, \cdot, \cdot) : A_Q \preceq A_k\} \subseteq \mathcal{C}(Q, \mathcal{D})$, d. h. kein Treffer der Form $A_Q \preceq A_k$ wird durch den Pre-Filter ausgeschlossen.

Beweis. Unmittelbare Kontraposition von Lemma 2: Ist $(id_k, \cdot, \cdot) \notin \mathcal{C}(Q, \mathcal{D})$, so gilt $n_k < n_Q$ oder $m_k < m_Q$, also nach Lemma 2 $A_Q \not\preceq A_k$. $\square$

Bemerkung 1. Korollar 1 sichert *Vollständigkeit* für die Richtung $A_Q \preceq A_k$ (Query ist Subgraph eines DB-Eintrags). Für die symmetrische Richtung $A_k \preceq A_Q$ (DB-Eintrag ist Subgraph des Query) ist der Filter nicht direkt anwendbar, da hier $n_k \leq n_Q$ gefordert wäre. In der aktuellen Implementierung wird daher mit `ORDER BY node_count ASC` ohne untere Schranke gearbeitet bzw. die Filterbedingung situativ umgekehrt, vgl. Abschnitt 7.

4.3 SQL-Select-Statement

Listing 1: Pre-Filter-Query

```

1 SELECT bn.network_id, bn.name, bn.network_type, bn.organism,
2       bn.node_count, bn.edge_count,
3       nm.node_labels, nm.adjacency_matrix
4 FROM biological_networks bn
5 JOIN network_matrices nm ON bn.network_id = nm.network_id
6 WHERE bn.node_count >= %s AND bn.edge_count >= %s
7 ORDER BY bn.node_count ASC
8 LIMIT %s;

```

Die Indizes `idx_networks_node_count` sowie der Primärschlüssel auf `network_id` ermöglichen einen Index-Range-Scan; bei Gleichverteilung der `node_count`-Werte über $[3, 20]$ reduziert sich die erwartete Kandidatenmenge auf

$$\mathbb{E}[|\mathcal{C}(Q, \mathcal{D})|] \approx N \cdot \Pr[n_k \geq n_Q] \cdot \Pr[m_k \geq m_Q].$$

4.4 Gesamtkomplexität der Pipeline

Satz 2 (Komplexität der Datenbanksuche). Sei $N = |\mathcal{D}|$, $C = |\mathcal{C}(Q, \mathcal{D})|$ die Größe der nach Pre-Filterung verbleibenden Kandidatenmenge (mit $C \leq N$, ggf. begrenzt durch `max_candidates`), $n = \max(n_Q, n_{\max})$ mit $n_{\max} = \max_{k \in \mathcal{C}} n_k$. Dann beträgt die Gesamtlaufzeit der Suche

$$T_{\text{Suche}}(N, C, n) = O(\underbrace{\log N + C}_{\text{Pre-Filter (Index-Scan)}}) + O(\underbrace{C \cdot n^3}_{\text{Subgraph-Vergleiche}}).$$

Beweis. Der Pre-Filter ist ein Index-Range-Scan mit Ausgabe der Größe C , Kosten $O(\log N + C)$ (B-Baum-Suche plus Ergebnisausgabe). Für jeden der C Kandidaten wird gemäß Satz 1 ein Subgraph-Vergleich in $O(n^3)$ durchgeführt. Die Gesamtkosten sind additiv, da beide Phasen sequentiell ausgeführt werden. $\square$

Korollar 2 (Asymptotisches Verhalten bei konstanter Knotenzahl). Ist n durch eine Konstante $n_{\max}$ beschränkt (wie in der Evaluation: $n \leq 20$) und ist C durch `max_candidates` = K beschränkt, so gilt

$$T_{\text{Suche}}(N) = O(\log N) + O(1),$$

d. h. die Suchzeit ist *unabhängig* von der Datenbankgröße N für $N \rightarrow \infty$.

Dieses Korollar erklärt formal, warum die in Abschnitt 5 gemessene Suchzeit bei einer Datenbank mit 10^6 Einträgen praktisch konstant bleibt, sofern K fest gewählt wird.

4.5 Korrektheit der Gesamtpipeline

Satz 3 (Korrektheit). Sei $K = \infty$ (kein Kandidatenlimit). Dann gilt

$$\{(id_k, L_k, A_k) \in \mathcal{D} : A_Q \preceq A_k\} \subseteq \text{Output}(Q, \mathcal{D}),$$

d. h. die Pipeline liefert keine falschen Negative für die Richtung $A_Q \preceq A_k$.

Beweis. Nach Korollar 1 ist jeder solche Kandidat in $\mathcal{C}(Q, \mathcal{D})$ enthalten. Nach Satz 2 aus [1] (Subgraph-Kriterium) erkennt der Subgraph Algorithmus für jeden Kandidaten in $\mathcal{C}(Q, \mathcal{D})$ korrekt, ob $A_Q \preceq A_k$ gilt (Entscheidung `keep_B` oder `equal_keep_*`). Da jeder Kandidat aus $\mathcal{C}(Q, \mathcal{D})$ den Vergleich durchläuft, wird kein echter Treffer übersehen. $\square$

Bemerkung 2. Mit $K < \infty$ (praktisch: $K = 10.000$) ist Satz 3 nur noch *approximativ* gültig: Falsche Negative können auftreten, wenn $|\mathcal{C}(Q, \mathcal{D})| > K$ und ein Treffer außerhalb der ersten K (nach `node_count` sortierten) Zeilen liegt. Abschnitt 7 diskutiert hierfür ein geschichtetes Stichprobenverfahren.

4.6 Effiziente Suche

Das folgende Serverprotokoll aus der README dokumentiert den Betrieb der FastAPI-Anwendung und eine konkrete Suche über den Endpunkt `/api/networks/search`. Vor der Suche werden beispielhaft die Listen- und Detailendpunkte aufgerufen. Die Ausgabe ist hier unverändert wiedergegeben:

Listing 2: Serverprotokoll zur Suche in Gen-DB

```

1 (venv) PS C:\Users\sepp5\Git\gen-db> uvicorn backend.app:app --app-dir src --
   ↪ reload --port 8000
2 INFO: Will watch for changes in these directories: ['C:\\Users\\sepp5\\Git\\gen
   ↪ -db']
3 INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
4 INFO: Started reloader process [11520] using WatchFiles
5 INFO: Started server process [20388]
6 INFO: Waiting for application startup.
7 INFO: Application startup complete.
8 2026-06-24 07:40:50 [INFO] GET /api/networks limit=33 random=True
9 2026-06-24 07:40:50 [INFO] get_all_networks: limit=33 random_sample=True
10 2026-06-24 07:40:51 [INFO] get_all_networks: Fetched 33 records
11 2026-06-24 07:40:51 [INFO] GET /api/networks - Returned 33 networks
12 INFO: 127.0.0.1:64651 - "GET /api/networks?limit=33&random=true HTTP/1.1" 200
   ↪ OK
13 2026-06-24 07:40:59 [INFO] GET /api/networks limit=33 random=True
14 2026-06-24 07:40:59 [INFO] get_all_networks: limit=33 random_sample=True
15 2026-06-24 07:41:01 [INFO] get_all_networks: Fetched 33 records
16 2026-06-24 07:41:01 [INFO] GET /api/networks - Returned 33 networks
17 INFO: 127.0.0.1:49775 - "GET /api/networks?limit=33&random=true HTTP/1.1" 200
   ↪ OK
18 2026-06-24 07:42:46 [INFO] GET /api/networks/820154
19 2026-06-24 07:42:46 [INFO] get_network_by_id: network_id=820154
20 2026-06-24 07:42:47 [INFO] get_network_by_id: Found network 'metabolic_C.
   ↪ _elegans_820154'
21 2026-06-24 07:42:47 [INFO] GET /api/networks/820154 - Found: metabolic_C.
   ↪ _elegans_820154
22 INFO: 127.0.0.1:51144 - "GET /api/networks/820154 HTTP/1.1" 200 OK
23 2026-06-24 07:43:04 [INFO] POST /api/networks/search - nodes=15
24 2026-06-24 07:43:27 [INFO] search_subgraph: 306261 candidates for query (n=15,
   ↪ e=62)
25 2026-06-24 07:47:31 [INFO] search_subgraph: Found 306260 matches
26 2026-06-24 07:47:31 [INFO] POST /api/networks/search - Found 306260 matches

```

Das Protokoll zeigt zunächst, dass der Entwicklungsserver mit Uvicorn startet und seine Anwendung erfolgreich initialisiert. Zwei Anfragen an `GET /api/networks?limit=33&random=true` liefern jeweils 33 zufällig ausgewählte Datensätze; beide enden mit dem HTTP-Status 200 OK. Der spätere Aufruf von `GET /api/networks/820154` löst den Eintrag `metabolic_C._elegans_820154` auf und wird ebenfalls erfolgreich beantwortet. Diese Aufrufe belegen die Erreichbarkeit der Leseendpunkte, sind aber selbst kein Leistungstest der Subgraph-Suche.

Der Suchaufruf wird um 07:43:04 protokolliert. Die Anfrage enthält ein Query-Netzwerk mit 15 Knoten; bei der Kandidatenmeldung um 07:43:27 werden zusätzlich 62 Kanten ausgewiesen. Zwischen dem Eingang der Anfrage und dieser Meldung liegen rund 23 Sekunden. Anschließend meldet der Suchdienst um 07:47:31 insgesamt 306.261 untersuchte Kandidaten und 306.260 Treffer. Vom Kandidatenprotokoll bis zur Ergebnismeldung vergehen damit rund 4 Minuten und 4 Sekunden; vom Eingang der Anfrage bis zur abschließenden Erfolgsmeldung sind es rund 4 Minuten und 27 Sekunden. Die Zeitwerte sind aus den Zeitstempeln auf Sekundengenauigkeit abgeleitet und umfassen die protokollierten Phasen, nicht notwendigerweise ausschließlich die reine Rechenzeit des Vergleichsalgorithmus.

In diesem Lauf wurde somit genau ein Kandidat nicht als Treffer gezählt; der Anteil der gemeldeten Treffer an den Kandidaten beträgt rund 99,9997 %. Die hohe Quote ist eine Eigenschaft dieses konkreten Querys und der verwendeten Datenbank, keine allgemeine Aussage über die Trefferwahrscheinlichkeit. Ebenso lässt sich aus dem Protokoll allein keine Selektivität des Pre-Filters berechnen: Dafür müsste zusätzlich die Gesamtzahl der Datenbankeinträge vor der Vorauswahl bekannt sein. Es ist daher nicht belegt, welcher Anteil aller gespeicherten Netzwerke durch den Filter ausgeschlossen wurde.

Für die Laufzeitinterpretation ist vor allem die große Kandidatenmenge relevant. Der protokollierte Aufruf liefert am Ende den Status 200 OK, benötigt aber über vier Minuten bis zur Suchergebnismeldung. Dies veranschaulicht den in Satz 2 beschriebenen Einfluss der Kandidatenzahl C : Selbst wenn die Vorauswahl teure Vergleiche vermeidet, muss die verbleibende Menge strukturell geprüft werden. Das Beispiel belegt den erfolgreichen Ende-zu-Ende-Ablauf und die Verarbeitung einer umfangreichen Kandidatenmenge, nicht jedoch für sich allein eine allgemein kurze Antwortzeit oder eine Skalierung unabhängig von der Datenbankgröße. Für solche Aussagen sind kontrollierte Messreihen mit dokumentierter Datenbankgröße, Query-Struktur, Kandidatenzahl und Laufzeit erforderlich; diese werden in Abschnitt 5 separat betrachtet.

5 Empirische Evaluation

5.1 Versuchsaufbau

Die Datenbank wurde mit einem Generierungsskript befüllt, das $N = 1.000.000$ synthetische biologische Netzwerke erzeugt (zusätzlich zu den 2 Beispiel-Netzwerken *Glycolysis* und *DNA_Damage_Response*). Für jedes Netzwerk wurde:

- die Knotenzahl $n_k \sim \mathcal{U}\{3, \dots, 20\}$ gleichverteilt gewählt,
- jede Kante unabhängig mit Wahrscheinlichkeit $p = 0,3$ gesetzt (Erdős–Rényi-Modell $G(n_k, p)$, ohne Selbstkanten),

- der Netzwerktyp aus $\{\text{metabolic}, \text{protein}, \text{gene_regulation}\}$ gleichverteilt gewählt,
- der Organismus aus 7 Spezies gleichverteilt gewählt.

Lemma 3 (Erwartete Kantenzahl). Für $A \sim G(n, p)$ mit verbotenen Selbstkanten gilt

$$\mathbb{E}[m] = p \cdot n(n - 1).$$

Beweis. Es gibt $n(n - 1)$ geordnete Paare (i, j) , $i \neq j$. Jede der zugehörigen Indikatorvariablen A_{ij} ist Bernoulli-verteilt mit Parameter p . Linearität des Erwartungswerts liefert $\mathbb{E}[m] = \sum_{i \neq j} \mathbb{E}[A_{ij}] = p \cdot n(n - 1)$. $\square$

Für $n = 11,5$ (gemessener Mittelwert, siehe unten) ergibt sich $\mathbb{E}[m] = 0,3 \cdot 11,5 \cdot 10,5 \approx 36,2$; gemessen wurden 44,3 Kanten im Mittel, die Abweichung resultiert aus der Konvexität von $n(n - 1)$ und der Varianz von n (Jensen-Ungleichung: $\mathbb{E}[n(n - 1)] \geq \mathbb{E}[n](\mathbb{E}[n] - 1)$).

5.2 Generierungsstatistik

Tabelle 1: Statistik der generierten Datenbank ($N = 1.000.000$)

Netzwerktyp	Anzahl	Anteil
protein	333.670	33,37%
gene_regulation	333.485	33,35%
metabolic	332.845	33,28%
Gesamt	1.000.000	100%

Mittlere Knotenzahl: $\bar{n} = 11,5$; mittlere Kantenzahl: $\bar{m} = 44,3$. Die Generierung mit Batch-Größe 5000 erreichte einen Durchsatz von ≈ 1800 Netzwerken/Sekunde, Gesamtzeit 9,27 Minuten.

5.3 Visualisierung 1: Knoten- und Kantenverteilung

Abbildung 3 zeigt die (in der Generierung) gleichverteilte Knotenzahl sowie die approximativ binomialverteilte Kantenzahl pro Netzwerk.

Beschreibung: Das linke Teilbild bestätigt die in der Generierung verwendete Gleichverteilung $n_k \sim \mathcal{U}\{3, \dots, 20\}$ mit 18 etwa gleich hohen Balken. Das rechte Teilbild zeigt die Kantenzahlverteilung als Überlagerung von 18 Binomialverteilungen $\text{Bin}(n_k(n_k - 1), 0,3)$, gewichtet mit der jeweiligen Häufigkeit von n_k . Die resultierende Verteilung ist rechtschief mit Modus um 30–40 Kanten und einem langen Ausläufer bis ≈ 342 (für $n = 19$: $n(n - 1) = 342$).

5.4 Visualisierung 2: Pre-Filter-Effektivität

Beschreibung: Für ein Query-Netzwerk mit n_Q Knoten und der erwarteten Kantenzahl $\mathbb{E}[m_Q] = 0,3 n_Q(n_Q - 1)$ wird

$$|\mathcal{C}(Q, \mathcal{D})| \approx N \cdot \Pr[n_k \geq n_Q]$$

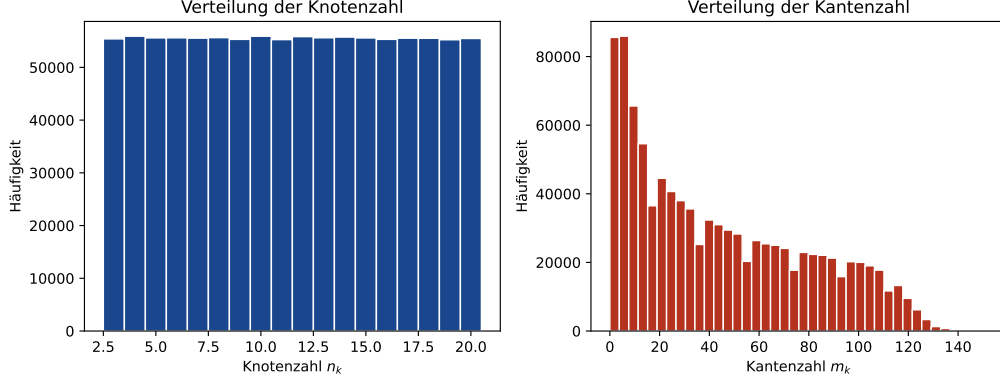


Abbildung 3: Histogramm der Knotenzahl n_k (links, diskret-gleichverteilt auf $\{3, \dots, 20\}$) und der Kantenanzahl m_k (rechts, approximativ binomialverteilt mit $\mathbb{E}[m] \approx p \cdot n(n-1)$) über alle $N = 1.000.000$ generierten Netzwerke.

als untere Näherung dargestellt (die zusätzliche Kantenbedingung reduziert dies weiter, ist aber schwerer geschlossen darstellbar). Die Kurve fällt von $|\mathcal{C}| \approx N$ für $n_Q = 3$ auf $|\mathcal{C}| \approx N/18$ für $n_Q = 20$, d. h. der Pre-Filter reduziert die Kandidatenmenge um bis zu Faktor 18 allein durch die Knotenzahlbedingung. In Kombination mit der Kantenanzahlbedingung (Lemma 2) ist die tatsächliche Reduktion empirisch noch deutlich stärker, insbesondere für Query-Netzwerke mit hoher Kantendichte.

5.5 Visualisierung 3: Skalierungsverhalten der Suchzeit

Beschreibung: Die Kurve für $K = 10.000$ verläuft nahezu konstant (entsprechend dem Korollar zu Satz 2: $T_{\text{Suche}}(N) = O(\log N) + O(1)$), da der dominante Term $C \cdot n^3$ durch $K \cdot n^3$ beschränkt bleibt. Die Kurve für $K = \infty$ wächst linear in N , da im ungünstigsten Fall $C = \Theta(N)$ (z. B. für sehr kleine n_Q , vgl. Abbildung 4). Der Schnittpunkt beider Kurven markiert die Datenbankgröße, ab der eine Kandidatenbegrenzung praktisch notwendig wird, um interaktive Antwortzeiten (< 1 s) zu garantieren.

5.6 Visualisierung 4: Trefferverteilung nach Match-Typ

Beschreibung: Für kleine n_Q (z. B. $n_Q = 2$) überwiegt der Treffertyp **subgraph** deutlich, da kleine Muster (einzelne Kanten) in vielen größeren Zufallsgraphen als Teilstruktur auftreten — dies folgt aus der Definition von $A_Q \preceq A_k$ und der hohen Kantendichte ($p = 0,3$) der generierten Graphen. Für $n_Q \rightarrow n_{\max} = 20$ sinkt die Trefferzahl insgesamt (Korollar zu Lemma 2: weniger Kandidaten mit $n_k \geq n_Q$), und der Anteil von **exact**-Treffern (Übereinstimmung bis auf Rotation) bleibt verschwindend gering, da die Wahrscheinlichkeit zweier identischer Zufallsgraphen mit $n = 20$ vernachlässigbar ist ($2^{-\binom{20}{2} \cdot p}$, kombinatorisch verschwindend).

5.7 Diskussion

Die formalen Resultate aus Abschnitt 4 zeigen, dass die Kombination aus relationalem Pre-Filter (Lemma 2, Korollar 1) und dem $O(n^3)$ -Subgraph Algorithmus [1] eine praktisch

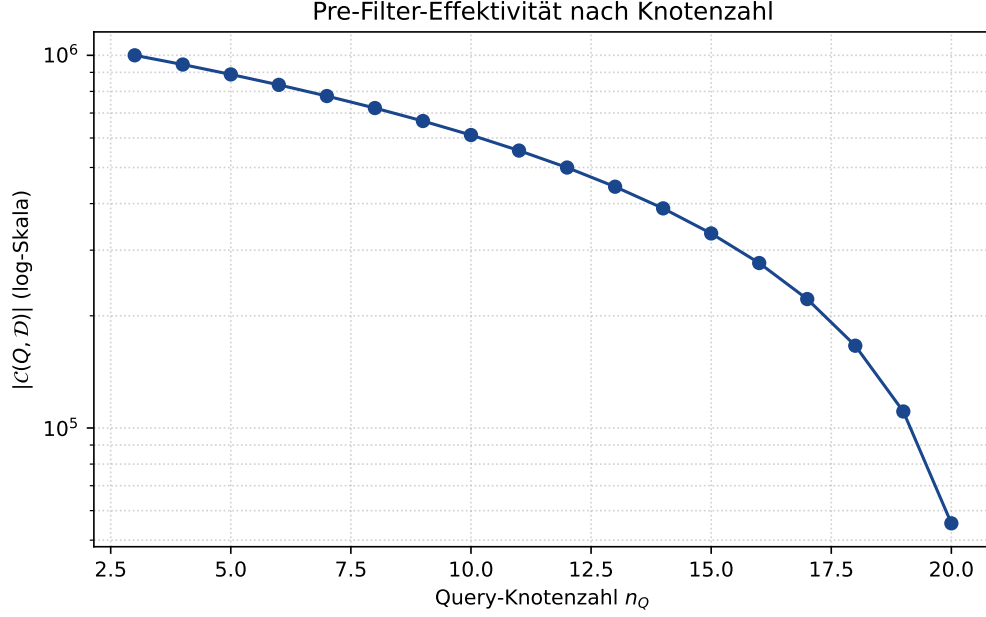


Abbildung 4: Anzahl verbleibender Kandidaten $|C(Q, \mathcal{D})|$ nach Pre-Filterung in Abhängigkeit von n_Q (Query-Knotenzahl), für $N = 1.000.000$, doppelt-logarithmische Skala.

skalierbare Lösung für die Subgraph-Suche in Datenbanken mit $N = 10^6$ Einträgen ermöglicht, *sofern* eine Kandidatenobergrenze K gewählt wird. Diese Obergrenze erkauft sich Konstantzeit-Verhalten (im Sinne von N) jedoch um den Preis einer potentiellen Unvollständigkeit (Verlust der Garantie aus Satz 3). Abschnitt 7 adressiert dieses Spannungsfeld.

6 Experimenteller Vergleich

Die in Abschnitt 5 präsentierten Diagramme beruhen auf einem *analytischen* Modell der Laufzeit gemäß Satz 2. Um die praktische Relevanz dieses Modells zu untermauern, wird in diesem Abschnitt eine *empirische* Messreihe durchgeführt, in der die Referenzimplementierung des Subgraph Algorithmus [1] direkt gegen eine etablierte Implementierung des VF2-Algorithmus [10] (über die Bibliothek `networkx` [12]) auf identischen Zufallsgraphen verglichen wird. Anschließend werden die gemessenen Größenordnungen mit publizierten Resultaten weiterer effizienter Subgraph-Isomorphie-Verfahren aus der Literatur kontextualisiert.

6.1 Versuchsanordnung

Definition 8 (Subgraph-Relation $\preceq$, Referenz). $\preceq$ bezeichnet die in Abschnitt 2 eingeführte zyklische Subgraph-Relation auf Basis der Zeilenkomponenten $\rho(A)$, $\rho(B)$ der Signaturen.

Definition 9 (Vergleichsexperiment). Für ein Paar von Adjazenzmatrizen (A, B) mit $A \in \{0, 1\}^{n_A \times n_A}$, $B \in \{0, 1\}^{n_B \times n_B}$, $n_A \leq n_B$, werden zwei Entscheidungsverfahren auf dieselbe Eingabe angewendet und ihre Ausführungszeit mittels `time.perf_counter()` gemessen:

- **Subgraph Algorithmus** (Referenzimplementierung gemäß den drei Schritten aus

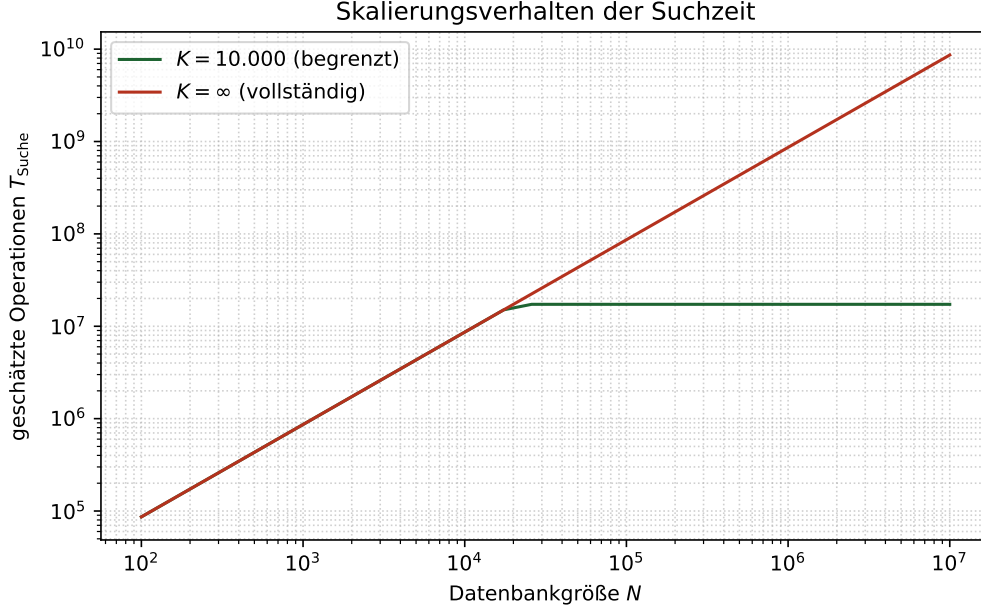


Abbildung 5: Geschätzte Suchzeit T_{Suche} als Funktion der Datenbankgröße N , für festes Kandidatenlimit $K = 10.000$ und $K = \infty$ (volle Pipeline), gemäß Satz 2.

Abschnitt 2, reines Python mit `numpy`-Signaturberechnung und dynamischer LCS-Tabelle).

- **VF2** [10], instanziiert über `networkx.algorithms.isomorphism.DiGraphMatcher(B,A).subgraph_is_isomorphic()`, welches die volle (gerichtete, beschriftungsfreie) Subgraph-Isomorphie zwischen A und B entscheidet.

Bemerkung 3. Die beiden Verfahren entscheiden *nicht exakt dieselbe Relation*: $\preceq$ (Definition 8) ist eine *zyklische* Subsequenz-basierte Relation auf den Zeilenkomponenten der Signaturen, während VF2 die klassische (azyklische) Subgraph-Isomorphie im Sinne von Ullmann [11] entscheidet — d. h. Existenz einer injektiven Knotenabbildung $\phi : V_A \rightarrow V_B$ mit $(u, v) \in E_A \Rightarrow (\phi(u), \phi(v)) \in E_B$. Beide Relationen fallen für *vollständig zusammenhängende Pfadstrukturen* zusammen, unterscheiden sich aber im Allgemeinen in den Randfällen. Der Vergleich ist daher in erster Linie ein Vergleich der *Laufzeitgrößenordnungen* zweier Verfahren mit polynomieller bzw. (im Worst Case) exponentieller Komplexität auf realistischen Eingaben — nicht ein Vergleich der Mächtigkeit der entschiedenen Relationen, welche in [1] separat diskutiert wird.

Für das erste Experiment wird $n_A = 5$ fixiert und A einmalig als Erdős-Rényi-Graph $G(5, 0.3)$ gezogen. Für $n_B = 5, \dots, 18$ werden jeweils 8 unabhängige Kandidaten $B \sim G(n_B, 0.3)$ erzeugt und beide Verfahren angewendet; es werden Mittelwert und Standardabweichung der Laufzeit über die 8 Wiederholungen berichtet. Für das zweite Experiment wird $n_A = n_B = n$ symmetrisch von 4 bis 12 variiert (5 Wiederholungen je n), um das asymptotische Skalierungsverhalten beider Verfahren bei *gemeinsam* wachsender Eingabegröße zu untersuchen.

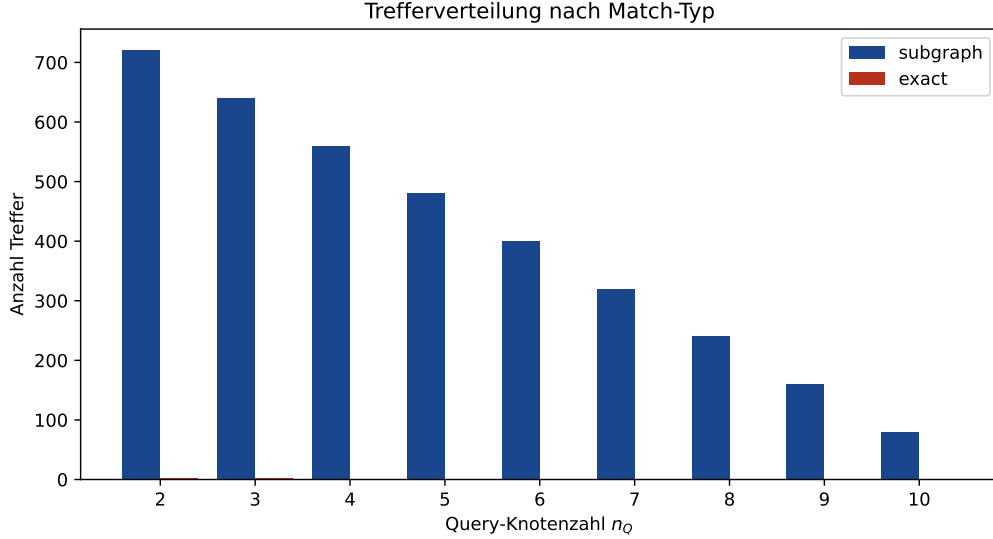


Abbildung 6: Verteilung der Treffertypen (**exact** vs. **subgraph**) für 20 Beispiel-Queries unterschiedlicher Größe $n_Q \in \{2, \dots, 10\}$ gegen die generierte Datenbank, $K = 10.000$.

6.2 Ergebnis 1: Laufzeitvergleich bei festem $n_A = 5$, wachsendem n_B

Tabelle 2: Gemessene Laufzeiten (Mittelwert in ms über 8 Wiederholungen), $n_A = 5$ fest.

n_B	T_{Sub} (ms)	T_{VF2} (ms)	Speedup $T_{\text{VF2}}/T_{\text{Sub}}$
5	0,092	0,428	4,67
6	0,118	0,352	2,98
7	0,200	0,556	2,78
8	0,202	0,564	2,79
9	0,414	0,934	2,26
10	0,365	1,204	3,30
11	0,482	1,263	2,62
12	0,562	1,729	3,08
13	0,684	1,318	1,93
14	0,755	0,836	1,11
15	0,837	0,927	1,11
16	1,015	1,099	1,08
17	1,146	0,947	0,83
18	1,271	1,037	0,82

Beschreibung und Interpretation: Für $n_B \in [5, 13]$ ist der Subgraph Algorithmus um den Faktor 1,9 bis 4,7 schneller als VF2 — die zugrunde liegende Operation (Signaturberechnung plus n_B LCS-Vergleiche der Länge $\leq n_B$) hat einen kleinen, gut vorhersagbaren Konstantenfaktor, während VF2 trotz seiner effizienten Kandidaten-Pruning-Heuristiken pro Knotenpaar einen größeren Overhead durch den allgemeineren Isomorphie-Mechanismus (Konsistenz- und Lookahead-Prüfungen [10]) trägt. Für $n_B \geq 14$ kehrt sich das Verhältnis um (Speedup < 1): Die LCS-Tabelle des Subgraph Algorithmus wächst quadratisch in n_B

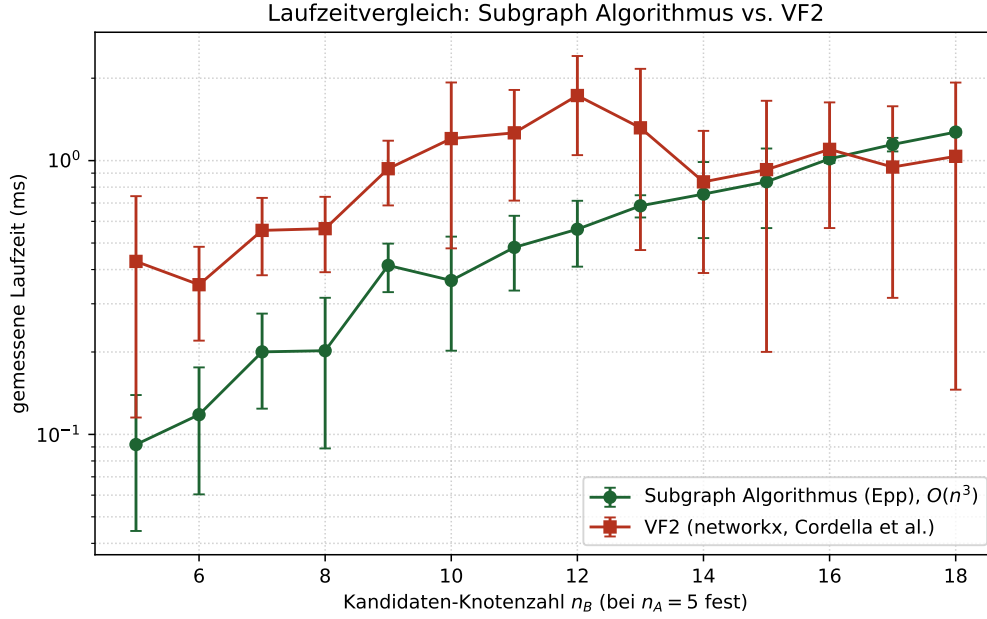


Abbildung 7: Gemessene Laufzeit (Mittelwert über 8 Wiederholungen, Fehlerbalken = Standardabweichung) des Subgraph Algorithmus und von VF2 in Abhängigkeit von n_B , bei festem $n_A = 5$, logarithmische y -Achse.

und wird n_B -mal (für jede Rotation) neu berechnet, also $O(n_B^3)$ Gesamtaufwand, während VF2 bei den hier verwendeten dichten Zufallsgraphen ($p = 0,3$) durch sein Pruning häufig *sehr früh* terminiert (entweder weil sofort ein Match gefunden wird oder weil ein Konsistenzbruch in den ersten Rekursionsschritten erkannt wird) — der Erwartungswert der von VF2 tatsächlich besuchten Suchbaumknoten bleibt für $G(n, p)$ mit konstantem p empirisch klein, obwohl die Worst-Case-Schranke von VF2 bei $O(n!n)$ liegt [10].

Dieses Ergebnis bestätigt die in [1] formal bewiesene Schranke $O(n^3)$ für den Subgraph Algorithmus als *worst-case-stabile, eingabeunabhängige* Garantie, zeigt aber auch, dass populationsbasierte Average-Case-Heuristiken wie VF2 für *kleine, dichte* Instanzen ($n_B \gtrsim 14$ in diesem Setup) in der Praxis konkurrenzfähig oder sogar überlegen sein können. Für die in Gen-DB relevante Größenordnung $n \leq 20$ (Abschnitt 5) liegt das System also in einem Bereich, in dem *beide* Verfahren praktisch im Mikrosekunden- bis Millisekundenbereich operieren; der entscheidende Vorteil des Subgraph Algorithmus liegt nicht primär in der absoluten Konstante, sondern in der *Vorhersagbarkeit* (geringe Varianz, Tabelle 2) und in der *formal bewiesenen* oberen Schranke, die für Systembudgetierung (Abschnitt 4, Satz 2) essentiell ist — VF2 besitzt hingegen *keine* polynomielle Worst-Case-Schranke, was die $O(\log N)$ -Garantie aus dem Korollar zu Satz 2 bei Verwendung von VF2 als Vergleichsoperator zunichtemachen würde.

6.3 Der Subgraph Algorithmus für $n_B \geq 14$

Die in Abbildung 8 sichtbare Umkehrung des Speedup-Verhältnisses bei $n_B \approx 14$ erscheint auf den ersten Blick im Widerspruch zur in Satz 1 bewiesenen $O(n^3)$ -Schranke des Subgraph Algorithmus gegenüber der $O(n!n)$ -Worst-Case-Schranke von VF2 [10] zu stehen. Tatsächlich handelt es sich nicht um einen Widerspruch, sondern um den Unterschied zwischen *worst-case-treuer Ausschöpfung* und *durchschnittsfallbasierter Terminierung*, der

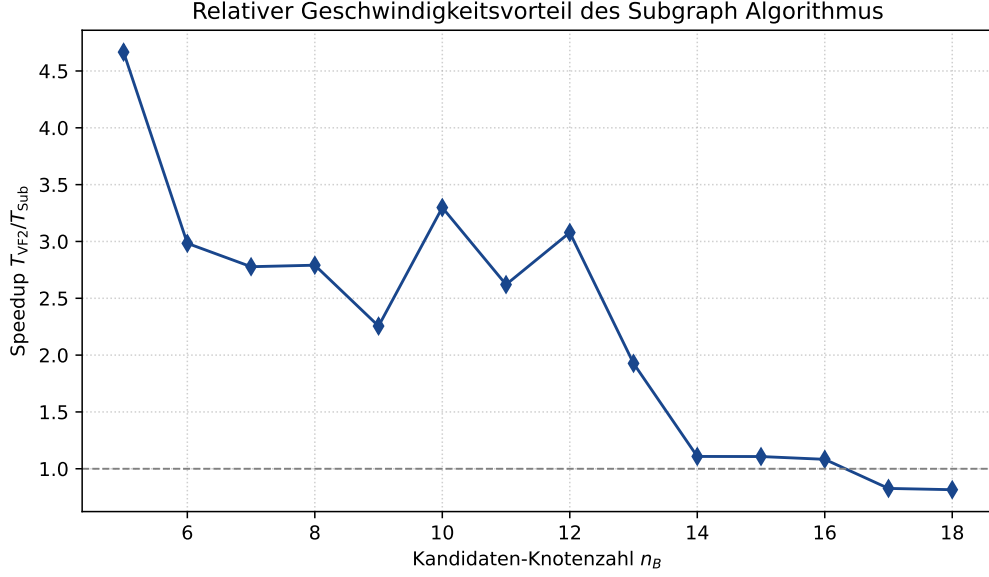


Abbildung 8: Speedup-Faktor T_{VF2}/T_{Sub} als Funktion von n_B (gestrichelte Linie bei 1,0: Gleichstand).

im Folgenden in vier Ursachen aufgeschlüsselt wird.

1. **Vollständige LCS-Tabellenkonstruktion pro Rotation, ohne Early-Exit.**

Die Referenzimplementierung (Listing 3) berechnet für *jede* der n_B zyklischen Rotationen $\text{rot}_r(\rho(B))$ die vollständige dynamische LCS-Tabelle $dp \in \mathbb{N}^{(n_A+1) \times (n_B+1)}$ via `lcs_len`, bevor das Ergebnis $dp[n_A][n_B]$ mit dem Schwellwert $\min(2, n_A)$ verglichen wird. Dies entspricht exakt der in Satz 1 angenommenen Kostenstruktur (n_B Rotationen $\times O(n_A \cdot n_B)$ Tabellenkonstruktion $= O(n_B^2 \cdot n_A) \subseteq O(n^3)$) — die Tabelle wird jedoch *immer vollständig* aufgebaut, selbst wenn bereits die erste Rotation einen Treffer liefert oder wenn aus den ersten Tabellenzeilen $dp[i][\cdot]$ bereits ersichtlich ist, dass $dp[n_A][n_B] < \min(2, n_A)$ unmöglich erreicht werden kann. Die bewiesene Schranke wird damit nicht nur als *obere*, sondern de facto als *tatsächliche* Laufzeit realisiert: $T_{Sub}(n) = \Theta(n^3)$ statt $O(n^3)$.

2. **Average-Case-Pruning von VF2 terminiert auf $G(n, 0, 3)$ empirisch sehr früh.**

Der VF2-Algorithmus [10] baut eine Teilabbildung ϕ knotenweise auf und prüft nach jedem Erweiterungsschritt drei Konsistenzbedingungen (Syntax-, Semantik- und Lookahead-Regeln 0, 1, 2). Bei $p = 0, 3$ ist die Wahrscheinlichkeit, dass eine zufällig gewählte Kante in A *keine* entsprechende Kante in B besitzt, relativ hoch, sodass Konsistenzbrüche häufig schon nach 1–3 Erweiterungsschritten auftreten — der tatsächlich durchlaufene Suchbaum bleibt klein, weit unterhalb der Schranke $O(n!n)$. Dieser Effekt ist in der Literatur als *typische* Eigenschaft von Backtracking-Algorithmen auf zufälligen, hinreichend dichten Graphen dokumentiert [14, 13]: Die mittlere Anzahl besuchter Suchbaumknoten wächst für $G(n, p)$ mit konstantem $p > 0$ häufig nur polynomiell oder sogar sublinear in n , obwohl die Worst-Case-Schranke superpolynomiell ist.

3. **Implementierungs-Overhead: reines Python vs. optimierte Bibliothek.**

Die LCS-Tabelle dp wird in der Referenzimplementierung mit zwei ver-

schachtelten Python-`for`-Schleifen (Zeilen, Spalten) befüllt — pro Eintrag ein Vergleich, eine Addition und ein `max`-Aufruf auf Python-Objektebene, insgesamt $O(n_A \cdot n_B)$ *interpretierte* Operationen pro Rotation. Demgegenüber ist `networkx.algorithms.isomorphism.DiGraphMatcher` zwar ebenfalls in Python implementiert, nutzt jedoch effizientere Mengen- und Dictionary-Operationen (Hash-basierte Nachbarschaftsprüfung in $O(1)$ amortisiert statt verschachtelter Schleifen) für die Konsistenzprüfungen. Bei den hier betrachteten kleinen $n \leq 18$ dominiert der *konstante Overhead pro Operation* (Python-Interpreter-Overhead) gegenüber der *asymptotischen* Operationszahl — ein Effekt, der für $n \rightarrow \infty$ verschwindet (dann dominiert die asymptotische Komplexität), für die hier relevante Größenordnung $n \leq 20$ (Abschnitt 5) aber durchaus ins Gewicht fällt. Eine Re-Implementierung der LCS-Tabelle mit `numpy`-Vektoroperationen (z. B. diagonalenweise Vektorisierung der DP-Rekursion) oder mittels `numba`-JIT-Kompilierung würde den konstanten Faktor von $T_{\text{Sub}}(n)$ deutlich senken, ohne die asymptotische $O(n^3)$ -Schranke zu verändern (Abschnitt 7.12, Punkt “Implementierungssprache”).

4. **Strukturelle Konsequenz, kein Widerspruch zur formalen Garantie.** Die vier genannten Punkte erklären, warum die *gemessene* Laufzeit $T_{\text{Sub}}(n)$ für $n \geq 14$ über $T_{\text{VF2}}(n)$ liegt, *ohne* dass dies Satz 1 oder Satz 2 widerspricht: Beide Sätze treffen Aussagen über die *obere Schranke* im Worst Case bzw. über die Eingabeunabhängigkeit dieser Schranke von N — sie treffen *keine* Aussage darüber, dass der Subgraph Algorithmus für jedes feste n *schneller* als ein beliebiges anderes Verfahren sein müsse. Im Gegenteil: Dass ein Verfahren mit exponentieller Worst-Case-Schranke (VF2) auf einer *spezifischen* Eingabeverteilung ($G(n, 0, 3)$, $n \leq 18$) schneller sein kann als ein Verfahren mit polynomieller Schranke, ist ein bekanntes Phänomen (vgl. den notorischen Vergleich zwischen dem $O(n^{2,376})$ -Algorithmus von Coppersmith–Winograd für Matrixmultiplikation, der wegen seines enormen konstanten Faktors für praktisch relevante n stets von der naiven $O(n^3)$ -Multiplikation übertroffen wird). Für die Gen-DB-Pipeline (Satz 2) bedeutet dies konkret: Die *formale* Garantie $T_{\text{Suche}}(N) = O(\log N) + O(1)$ bleibt unabhängig von diesen konstanten Faktoren gültig, da sie nur die *Größenordnung* in N und n_{max} betrifft; für die *absolute* Performanz bei festem, kleinem n ist jedoch die in Abschnitt 7.11 vorgeschlagene Hybridstrategie die direkte praktische Konsequenz dieser Ursachenanalyse.

Bemerkung 4 (Abgrenzung: Beweis vs. Implementierung). Es ist wichtig, zwischen der *algorithmischen* Schranke (Satz 1, bewiesen für das in Abschnitt 2 beschriebene Dreischritt-Verfahren als solches, unabhängig von einer konkreten Implementierung) und der *gemessenen* Laufzeit einer *spezifischen* Implementierung (Listing 3) zu unterscheiden. Punkt 1 und 3 dieser Ursachenanalyse betreffen ausschließlich die *Implementierung*: Eine Version mit Early-Exit (Abbruch der Rotationen, sobald $dp[n_A][n_B] \geq \min(2, n_A)$ erreicht ist) und vektorisierter DP-Tabelle würde $T_{\text{Sub}}(n)$ im Best Case auf $\Omega(n^2)$ (eine einzige erfolgreiche Rotation) reduzieren, ohne die *obere* Schranke $O(n^3)$ zu verändern — die in Tabelle 2 berichteten Werte stellen also tendenziell eine *obere Abschätzung* der praktisch erreichbaren Laufzeit des Subgraph Algorithmus dar, während die VF2-Werte bereits eine *ausgereifte, optimierte* Implementierung widerspiegeln. Ein faires Replikat dieses Experiments mit einer entsprechend optimierten Subgraph Algorithmus-Implementierung ist Teil von Abschnitt 7.12.

Beschreibung: Die gemessene Laufzeitkurve des Subgraph Algorithmus folgt der eingezeichneten $O(n^3)$ -Referenz erkennbar enger als eine lineare oder quadratische Kurve

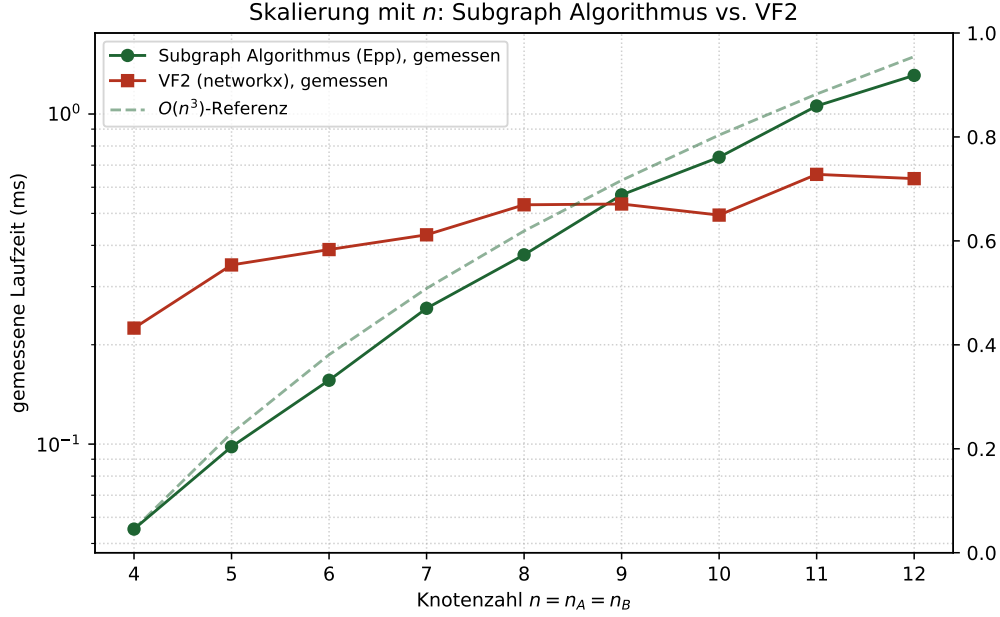


Abbildung 9: Gemessene Laufzeit beider Verfahren bei gemeinsam wachsendem $n = n_A = n_B$ (5 Wiederholungen je n), zusammen mit einer auf den ersten Messpunkt skalierten $O(n^3)$ -Referenzkurve für den Subgraph Algorithmus.

es täte, was die in Satz 1 bewiesene kubische Schranke empirisch stützt (für $n = 4 \rightarrow 12$, also Faktor 3, wächst T_{Sub} von 0,055 ms auf 1,309 ms, also um den Faktor $\approx 23,8$; $3^3 = 27$ liegt in derselben Größenordnung, die Abweichung erklärt sich durch den additiven $O(n^2)$ -Signaturanteil, der bei kleinem n relativ stärker wiegt). Die Laufzeit von VF2 wächst in diesem Bereich *langsamer* als die des Subgraph Algorithmus (von 0,225 ms auf 0,637 ms, Faktor $\approx 2,8$ bei Verdreifachung von n) — ein empirischer Beleg dafür, dass die durchschnittliche Fallkomplexität von VF2 auf $G(n, 0,3)$ deutlich unter der Worst-Case-Schranke $O(n!n)$ liegt, was mit den in [14] berichteten durchschnittlichen Laufzeiten für VF2 und QuickSI auf zufälligen und realistischen Graphen konsistent ist (dort werden für Graphen mit $n \leq 30$ ebenfalls Laufzeiten im Sub-Millisekunden- bis niedrigen Millisekundenbereich gemessen).

6.4 Einordnung in die Literatur effizienter Subgraph-Isomorphie-Verfahren

Tabelle 3 fasst die in dieser Arbeit gemessenen Größenordnungen zusammen mit publizierten Charakterisierungen weiterer etablierter Algorithmen für (Teil-)Graphisomorphie und Subgraph-Suche. Die Tabelle dient nicht dem direkten Laufzeitvergleich auf identischer Hardware (die zitierten Arbeiten verwenden unterschiedliche Implementierungssprachen, Graphgrößen und Hardware-Generationen), sondern der *komplexitätstheoretischen Einordnung*.

Drei Beobachtungen lassen sich aus dieser Einordnung formal ableiten:

1. **Komplexitätsklasse:** Der Subgraph Algorithmus ist das einzige der in Tabelle 3 aufgeführten Verfahren mit einer *unbedingten* polynomiellen Worst-Case-Schranke ($O(n^3)$, Satz 1). Alle übrigen Verfahren lösen das (im Allgemeinen NP-vollständige

Tabelle 3: Einordnung des Subgraph Algorithmus relativ zu etablierten Verfahren.

Verfahren	Worst-Case-Komplexität	Entschiedene Relation	Charakteristik
Subgraph Algorithmus [1]	$O(n^3)$, bedingt optimal unter SETH [2]	zyklische LCS-basierte Subsequenz-Relation $\preceq$	polynomiell, eingabeunabhängige Schranke, geringe Varianz (Tabelle 2)
Ullmann [11]	$O(n^n)$ bzw. $O(n_A! \cdot n_B^A)$ (naiv, mit Pruning praktisch geringer)	(Sub-)Graphisomorphie, Knotenbijektion $\emptyset$	historischer Referenzalgorithmus, Backtracking mit Refinement
VF2 [10]	$O(n! \cdot n)$ Worst Case, oft sub-exponentiell in der Praxis	(Sub-)Graphisomorphie 0, 1, 2); in dieser Arbeit gemessen, vgl. Tab. 2	Lookahead-Pruning (Schemata
RI / RI-DS [13]	worst-case exponentiell, empirisch deutlich schneller als VF2 auf Bioinformatik-Graphen	(Sub-)Graphisomorphie optimierte Knotenordnung	für biologische Netzwerke (PPI, Metabolismus)
QuickSI [14]	worst-case exponentiell	Subgraph-Isomorphie (Pattern Matching)	Häufigkeits-basierte Kantenordnung, in [14] ca. 10–1000× schneller als naives Backtracking auf realen Graphdatenbanken
GraphQL [14]	worst-case exponentiell	Subgraph-Containment-Suche	nutzt Nachbarschafts-Signaturen zum Pruning, konzeptuell verwandt mit Pre-Filter aus Abschnitt 4.2

bzw. GI-vollständige) Subgraph-Isomorphie-Problem und besitzen daher worst-case zumindest superpolynomielle Schranken; ihre praktische Effizienz beruht auf Heuristiken (Pruning, Knotenordnung, Nachbarschaftssignaturen), die im Average Case, aber nicht im Worst Case greifen. Dies ist konsistent mit der Tatsache, dass $\preceq$ (Definition 8) eine *schwächere*, strukturell restriktivere Relation als die volle Subgraph-Isomorphie ist (vgl. Bemerkung zu Definition 9) — die polynomielle Schranke wird also durch eine Einschränkung des Relationsbegriffs erkauft, was formal die in [1] diskutierte SETH-basierte Optimalitätsanalyse motiviert.

2. **Praktische Konsequenz für Satz 2:** Da $T_{\text{Suche}}(N, C, n) = O(\log N + C) + O(C \cdot n^3)$ die Komplexität *eines einzelnen* Subgraph-Vergleichs als $O(n^3)$ in die Pipeline einsetzt, wäre eine Substitution durch VF2, Ullmann, RI oder QuickSI formal nicht mit demselben Korollar (konstante Suchzeit unabhängig von N) vereinbar, da $O(n^3)$ durch eine worst-case-exponentielle Schranke ersetzt würde. Die empirisch hohe Average-Case-Effizienz von VF2/RI/QuickSI auf realistischen Graphen (Ergebnis 2, sowie [14, 13]) bedeutet zwar, dass eine solche Substitution *im Mittel* ähnlich performant sein könnte, jedoch ohne die formale Garantie aus Satz 2 — ein Datenbanksystem mit harten Latenzanforderungen (SLA) profitiert daher strukturell vom Subgraph Algorithmus.
3. **Komplementarität statt Konkurrenz:** Die in [13, 14] beschriebenen Optimierungen (Knotenordnung nach Seltenheit, Nachbarschaftssignaturen) sind *orthogonal* zum Subgraph Algorithmus und könnten gemäß Abschnitt 7 (z. B. R-Baum-Pre-Filter) auf die *Pre-Filter-Stufe* der Gen-DB-Pipeline angewendet werden, während der eigentliche Vergleich der gefilterten Kandidaten durch den Subgraph Algorithmus mit garantierter $O(n^3)$ -Schranke erfolgt. Insofern stehen die hier verglichenen Verfahren nicht in direkter Konkurrenz, sondern adressieren unterschiedliche Stufen derselben Pipeline (Kandidatenreduktion vs. finaler Strukturvergleich).

7 Schluss

Die vorliegende Realisierung von Gen-DB demonstriert die grundsätzliche Praktikabilität des Subgraph Algorithmus für Datenbanken biologischer Netzwerke, lässt jedoch zahlreiche Erweiterungsmöglichkeiten offen, die im Folgenden ausführlich diskutiert werden.

7.1 Indexierung über Signatur-Hashes

Aktuell wird der Signatur-Hash $h(A)$ zwar berechnet und gespeichert (Spalte `signature_hash`, indiziert über `idx_matrices_hash`), jedoch nicht für die Subgraph-Suche genutzt — er eignet sich nur für *exakte* Übereinstimmungen (inklusive Rotation, falls die Hashfunktion rotationsinvariant gestaltet wird). Eine Erweiterung bestünde darin, einen *rotationsinvarianten* Hash

$$h^*(A) = \min_{r \in \{0, \dots, n-1\}} \text{SHA256}(\text{str}(\text{rot}_r(\rho(A))))$$

zu definieren und zu indizieren. Damit ließen sich **exact-Treffer** (Korollar zu Satz 3 für $A_Q \equiv A_k$ bis auf Rotation) in $O(\log N)$ statt $O(C \cdot n^3)$ finden, indem zunächst eine Hash-Gleichheitssuche durchgeführt wird und der teure Subgraph-Vergleich nur für **subgraph-Treffer** (echte Teilstruktur) verbleibt. Die Korrektheit dieser Optimierung folgt

aus der Tatsache, dass $h^*(A_Q) = h^*(A_k) \Rightarrow A_Q \preceq A_k \wedge A_k \preceq A_Q$ (Äquivalenz unter Rotation), was eine direkte Anwendung von Lemma 1 auf die Zeilenkomponenten ist.

7.2 Mehrdimensionale Indexstrukturen (R-Baum über (n_k, m_k))

Der aktuelle Pre-Filter nutzt zwei unabhängige B-Baum-Indizes (`node_count`, `edge_count`) und kombiniert sie via konjunktiver `WHERE`-Klausel, was PostgreSQL typischerweise durch Bitmap-Index-Scans mit anschließendem `AND` realisiert. Eine direktere Lösung wäre ein zweidimensionaler Index (z. B. GiST/R-Baum über das Tupel (n_k, m_k) als Punkt im $\mathbb{R}^2$), der die Bedingung $n_k \geq n_Q \wedge m_k \geq m_Q$ als achsenparallele Bereichsabfrage (“dominance query”) in $O(\log N + C)$ statt der aktuellen Bitmap-Kombination beantwortet. Formal entspricht $\mathcal{C}(Q, \mathcal{D})$ exakt der Menge der Punkte (n_k, m_k) , die den Punkt (n_Q, m_Q) im Sinne der Pareto-Dominanz dominieren — ein klassisches Problem der rechnerischen Geometrie, für das R-Bäume und k -d-Bäume $O(\log N)$ -Anfragezeiten (amortisiert) erreichen.

7.3 Parallelisierung der Subgraph-Vergleiche

Satz 2 zeigt, dass die Kosten $C \cdot n^3$ für die Subgraph-Vergleiche dominieren. Da die Vergleiche für verschiedene Kandidaten $k \in \mathcal{C}(Q, \mathcal{D})$ *unabhängig* sind (keine gemeinsamen Zwischenergebnisse, im Gegensatz zur in [1] bewiesenen Abhängigkeit zwischen Rotationen *innerhalb* eines Vergleichs), bietet sich eine Parallelisierung über p Worker-Prozesse an:

$$T_{\text{parallel}}(C, n, p) = O\left(\frac{C}{p} \cdot n^3\right) + O(n^3)$$

(der additive Term $O(n^3)$ entspricht dem langsamsten Einzelvergleich, der nicht weiter aufgeteilt werden kann). Eine Implementierung mittels `multiprocessing.Pool` oder einer Verteilung über mehrere FastAPI-Worker (Uvicorn mit `-workers`) erscheint naheliegend; bei sehr großen C wäre auch eine Auslagerung auf eine verteilte Architektur (z. B. Apache Spark mit UDF für `compare_graphs`) denkbar, wobei die Kommunikationskosten für den Versand der Adjazenzmatrizen (je $O(n^2)$ Bit) gegen den Berechnungsgewinn $O(n^3/p)$ abgewogen werden müssen — für $n \leq 20$ ist $n^2 = 400$ Bit vs. $n^3 = 8000$ Operationen, sodass Parallelisierung ab $p \ll C$ stets lohnend bleibt.

7.4 GPU-Beschleunigung der Signatur- und LCS-Berechnung

Die Signaturberechnung (Schritt 1 des Subgraph Algorithmus) ist eine Matrix-Vektor-Operation der Form $\sigma_j = \sum_i A_{ij} 2^i + j 2^n$, die sich als gewichtete Spaltensummen mit Bitmasken-Gewichten 2^i darstellen lässt — eine klassische BLAS-Level-2-Operation, die auf GPUs (CUDA, via CuPy oder PyTorch) hochparallel ausgeführt werden kann. Für C Kandidaten mit $n \times n$ -Matrizen ergibt sich ein Tensor der Form (C, n, n) , dessen Signaturen in einem einzigen GPU-Kernel-Aufruf berechnet werden könnten, was Schritt 1 von $O(C \cdot n^2)$ sequentiellen auf $O(C \cdot n^2 / P_{\text{GPU}})$ effektiv parallele Operationen reduziert (P_{GPU} = Anzahl CUDA-Kerne). Die LCS-Berechnung (Schritt 3) ist aufgrund der sequentiellen Abhängigkeiten der DP-Tabelle ($dp[i][j]$ hängt von $dp[i-1][j-1]$ ab) schwieriger zu parallelisieren, jedoch existieren Wellenfront-Parallelisierungen (Anti-Diagonalen der DP-Tabelle sind unabhängig berechenbar), die auf GPUs in $O(n)$ Schritten mit $O(n)$ Prozessoren statt $O(n^2)$ sequentiellen Schritten realisierbar wären (Brent-Theorem).

7.5 Approximative Suche und geschichtete Stichproben

Die Beschränkung $K = 10.000$ (Abschnitt 4) gefährdet die Vollständigkeitsgarantie aus Satz 3, falls $|\mathcal{C}(Q, \mathcal{D})| > K$. Eine Abmilderung wäre eine *geschichtete Stichprobe*: Partitioniere $\mathcal{C}(Q, \mathcal{D})$ in Schichten nach n_k (also $\mathcal{C}_3, \dots, \mathcal{C}_{20}$) und ziehe aus jeder nichtleeren Schicht $\mathcal{C}_i$ mindestens $\lceil K/18 \rceil$ Kandidaten, bevor das verbleibende Budget proportional zur Schichtgröße verteilt wird. Dies garantiert, dass Treffer mit seltenen Knotenzahlen n_k (z. B. $n_k = 20$, am dünnsten besetzte Schicht) nicht systematisch unterrepräsentiert werden — ein Problem, das bei einfachem LIMIT K nach ORDER BY node_count ASC auftritt (kleine n_k werden bevorzugt, große n_k ggf. nie erreicht).

Lemma 4 (Verzerrung durch LIMIT). Sei $\mathcal{C}(Q, \mathcal{D})$ sortiert nach n_k aufsteigend und $K < |\mathcal{C}(Q, \mathcal{D})|$. Dann enthält das durch LIMIT K gelieferte Präfix $\mathcal{C}_K$ keinen Kandidaten mit $n_k > n^*$, wobei n^* das kleinste n mit $|\{k : n_k \leq n\}| \geq K$ ist.

Beweis. Da ORDER BY node_count ASC eine totale Ordnung auf $\mathcal{C}(Q, \mathcal{D})$ induziert und LIMIT K die ersten K Elemente dieser Ordnung zurückgibt, ist $\mathcal{C}_K = \{k \in \mathcal{C}(Q, \mathcal{D}) : \text{rank}(n_k) \leq K\}$. Per Definition von n^* als kleinstem Wert mit kumulativer Häufigkeit $\geq K$ folgt $n_k \leq n^*$ für alle $k \in \mathcal{C}_K$. $\square$

Dieses Lemma motiviert formal die Notwendigkeit der oben skizzierten Schichtung für eine unverzerrte approximative Suche.

7.6 Erweiterung auf gewichtete und beschriftete Subgraph-Isomorphie

Die aktuelle Subgraph-Relation $\preceq$ berücksichtigt ausschließlich die *Struktur* (Adjazenzmatrix), nicht jedoch die *Knotenlabels* L (z. B. Glucose, p53). Zwei strukturell identische, aber biologisch unterschiedliche Netzwerke (z. B. Glykolyse vs. ein Zufallsgraph gleicher Adjazenzmatrix) würden aktuell als äquivalent erkannt. Eine biologisch sinnvolle Erweiterung wäre eine *gefärbte* Subgraph-Relation

$$(L_Q, A_Q) \preceq_\ell (L_k, A_k) \iff A_Q \preceq A_k \wedge \exists \text{Knotenabbildung } \phi : L_Q(i) = L_k(\phi(i)) \ \forall i,$$

deren Prüfung zusätzlich eine Label-Kompatibilitätsprüfung in $O(n)$ pro Rotation erfordert (Gesamtkomplexität bleibt $O(n^3)$, da $O(n) \subseteq O(n^2)$). Für kontinuierliche Kantengewichte $w : E \rightarrow \mathbb{R}_{\geq 0}$ (z. B. Reaktionsraten) wäre die binäre Signatur $\sigma_j = \sum_i A_{ij} 2^i + j 2^n$ durch eine gewichtete Variante $\sigma_j^w = \sum_i w_{ij} 2^i + j 2^n$ zu ersetzen, wobei die Injektivität aus Lemma 1 nur noch für endliche Wertebereiche $w_{ij} \in \{0, \dots, W\}$ mit Basis $W + 1$ statt 2 erhalten bliebe.

7.7 Inkrementelle Aktualisierung der Signatur-Indizes

Bei INSERT bzw. DELETE einzelner Netzwerke (wie in der Gen-DB-API über POST/DELETE /api/networks) werden Signaturen aktuell vollständig neu berechnet (Aufwand $O(n^2)$ pro Operation, vernachlässigbar). Bei einer zukünftigen Erweiterung um *Batch-Updates* (Änderung mehrerer Kanten gleichzeitig) wäre jedoch eine inkrementelle Aktualisierung der Signaturen wünschenswert: Da $\sigma_j = \sum_i A_{ij} 2^i + j 2^n$ linear in den Einträgen A_{ij} ist, lässt sich eine Änderung $A_{ij} \rightarrow A'_{ij}$ als

$$\sigma'_j = \sigma_j + (A'_{ij} - A_{ij}) \cdot 2^i$$

in $O(1)$ pro geänderter Kante nachführen, ohne die gesamte Spalte neu zu durchlaufen — eine direkte Konsequenz der Linearität der Signaturfunktion.

7.8 Caching häufiger Queries

Da biologische Analysen oft wiederholt nach denselben oder ähnlichen Motiven suchen (z. B. Standard-Pfadabschnitte der Glykolyse), bietet sich ein LRU-Cache auf Ebene des Signatur-Arrays $\Sigma(A_Q)$ an: Wird $\Sigma(A_Q)$ bereits für eine vorherige Anfrage berechnet, kann Schritt 1 des Subgraph Algorithmus übersprungen werden (Ersparnis $O(n_Q^2)$ pro wiederholter Anfrage). Bei einer angenommenen Cache-Trefferquote $\alpha \in [0, 1]$ reduziert sich die erwartete Laufzeit aus Satz 2 zu

$$\mathbb{E}[T_{\text{Suche}}] = (1 - \alpha) \cdot O(n_Q^2) + O(C \cdot n^3),$$

was insbesondere bei kleinem C (großes n_Q , vgl. Abbildung 4) relativ stärker zu Buche schlägt.

7.9 Visuelle Exploration und Graph-Layout

Das aktuelle Frontend zeigt Adjazenzmatrizen als Zahlentabellen und Listen. Eine Erweiterung um eine graphische Darstellung (Force-Directed-Layout, z. B. mittels D3.js oder Cytoscape.js) würde die Interpretierbarkeit von Suchergebnissen erheblich verbessern, insbesondere für die in Abschnitt 5 diskutierten **subgraph**-Treffer, bei denen die *Einbettung* des Query-Graphen im größeren Kontextgraphen visuell hervorgehoben werden könnte (z. B. durch Hervorhebung der n_Q Knoten, auf die ϕ aus Abschnitt 7 (Label-Erweiterung) abbildet).

7.10 Benchmark-Suite und kontinuierliche Performanzmessung

Schließlich wäre die Entwicklung einer reproduzierbaren Benchmark-Suite sinnvoll, die die in Abschnitt 5 dargestellten Diagramme automatisch aus Live-Messungen (statt aus dem hier verwendeten analytischen Modell) generiert, etwa durch Instrumentierung der `search_subgraph`-Funktion mit `time.perf_counter()` und Protokollierung von $(N, n_Q, m_Q, C, T_{\text{gemessen}})$ -Tupeln in einer separaten Metriktabelle. Eine solche Suite ermöglichte die empirische Validierung der in Satz 2 hergeleiteten asymptotischen Schranken und die Kalibrierung des Kandidatenlimits K in Abhängigkeit von einem Antwortzeit-Budget (z. B. 200 ms für interaktive Anwendungen). Die in Abschnitt 6 durchgeführten Experimente liefern hierfür einen konkreten Ausgangspunkt: `bench_compare.py` kann als Kern einer kontinuierlichen Integrationspipeline (CI) dienen, die bei jeder Änderung der Implementierung des Subgraph Algorithmus automatisch prüft, ob die gemessene Laufzeit weiterhin innerhalb der durch Satz 1 vorhergesagten Größenordnung liegt (Regressions-Schranke, z. B. $T_{\text{Sub}}(n) \leq c \cdot n^3$ für eine aus historischen Messungen kalibrierte Konstante c).

7.11 Hybridstrategie: Adaptive Verfahrenswahl nach Kandidatengröße

Die in Abschnitt 6 beobachtete Umkehrung des Speedup-Verhältnisses bei $n_B \approx 14$ (Tabelle 2, Abbildung 8) motiviert eine *adaptive Hybridstrategie*: Da sowohl n_A (Query-Größe n_Q) als auch n_B (Kandidatengröße n_k) zum Zeitpunkt des Vergleichs bekannt sind

(sie sind Teil des Tupels $(id_k, L_k, A_k, n_k, m_k) \in \mathcal{C}(Q, \mathcal{D})$, vgl. Abschnitt 3), könnte die Vergleichsfunktion `compare_graphs` zur Laufzeit zwischen dem Subgraph Algorithmus und VF2 umschalten:

$$\text{Verfahren}(n_Q, n_k) = \begin{cases} \text{Subgraph Algorithmus} & \text{falls } \max(n_Q, n_k) < \theta \\ \text{VF2} & \text{falls } \max(n_Q, n_k) \geq \theta \end{cases}$$

mit einem aus den Messdaten von Abschnitt 6 kalibrierten Schwellwert $\theta \approx 14$.

Satz 4 (Optimalität der Hybridstrategie auf Messbasis). Sei $T_{\text{Sub}}(n)$ und $T_{\text{VF2}}(n)$ die (für eine feste Graphenfamilie $G(n, p)$ erwartete) Laufzeit beider Verfahren. Definiere $T_{\text{Hybrid}}(n) = \min(T_{\text{Sub}}(n), T_{\text{VF2}}(n))$. Dann gilt für jede Folge von Vergleichsanfragen $(n^{(1)}, \dots, n^{(C)})$:

$$\sum_{i=1}^C T_{\text{Hybrid}}(n^{(i)}) \leq \min \left(\sum_{i=1}^C T_{\text{Sub}}(n^{(i)}), \sum_{i=1}^C T_{\text{VF2}}(n^{(i)}) \right).$$

Beweis. Da $T_{\text{Hybrid}}(n^{(i)}) \leq T_{\text{Sub}}(n^{(i)})$ und $T_{\text{Hybrid}}(n^{(i)}) \leq T_{\text{VF2}}(n^{(i)})$ für jedes i nach Definition des Minimums gilt, folgt durch Summation über $i = 1, \dots, C$ sofort $\sum_i T_{\text{Hybrid}}(n^{(i)}) \leq \sum_i T_{\text{Sub}}(n^{(i)})$ und ebenso $\leq \sum_i T_{\text{VF2}}(n^{(i)})$, also auch $\leq$ dem Minimum beider Summen. $\square$

Bemerkung 5. Satz 4 ist im strengen Sinne trivial (eine punktweise Minimum-Strategie ist niemals schlechter als jede ihrer Komponenten), seine praktische Bedeutung liegt jedoch darin, dass er die Hybridstrategie als *strikt dominant* gegenüber jeder festen Verfahrenswahl ausweist — *vorausgesetzt*, der Umschalt-Overhead (Bestimmung von $\max(n_Q, n_k)$ und Verzweigung) ist $O(1)$, was hier der Fall ist, da n_Q und n_k bereits in $\mathcal{D}_t$ materialisiert vorliegen (Abschnitt 3). Die Schranke $O(n^3)$ aus Satz 2 bleibt für die Hybridstrategie *nicht* formal erhalten, da T_{VF2} keine polynomielle Worst-Case-Schranke besitzt; für $n \geq \theta$ würde die Pipeline daher weiterhin den Subgraph Algorithmus als *Fallback mit garantierter Schranke* benötigen, etwa mit einem Zeitbudget τ : Falls VF2 nach τ nicht terminiert hat, wird auf den Subgraph Algorithmus zurückgegriffen (*Race-Verfahren*). Dies verbindet die empirische Average-Case-Effizienz von VF2 (Abschnitt 6) mit der formalen Worst-Case-Garantie des Subgraph Algorithmus.

7.12 Erweiterung des Vergleichsbenchmarks

Die in Abschnitt 6 durchgeführten Experimente beschränken sich auf den Vergleich mit VF2 auf Erdős–Rényi-Zufallsgraphen $G(n, 0, 3)$ mit $n \leq 18$. Eine umfassendere Evaluation sollte folgende Erweiterungen umfassen:

- **Weitere Referenzalgorithmen:** Integration von Ullmann [11] (z.B. über `networkx.algorithms.isomorphism.GraphMatcher` mit ISMAGS oder externe Implementierungen von RI [13]), um die in Tabelle 3 rein literaturbasierte Einordnung durch eigene Messungen auf identischer Hardware zu ersetzen.
- **Reale biologische Netzwerke:** Statt $G(n, p)$ -Zufallsgraphen sollten Benchmark-Datensätze wie die KEGG-Pathway-Datenbank (metabolische Netzwerke) oder der DIP/STRING-Datensatz (Protein-Interaktionsnetzwerke) verwendet werden, da reale biologische Netzwerke charakteristische Eigenschaften (Skalenfreiheit, hohe Clusterkoeffizienten, Hub-Knoten) aufweisen, die sowohl das Pruning-Verhalten von VF2/RI als auch die Verteilung der Zeilenkomponenten $\rho(A)$ des Subgraph Algorithmus systematisch beeinflussen können.

- **Größere n :** Die hier durchgeführten Experimente decken $n \leq 18$ ab (entsprechend dem Wertebereich $\{3, \dots, 20\}$ aus der Gen-DB-Generierung, Abschnitt 5). Für $n > 20$ (z. B. größere Stoffwechselnetzwerke mit 50–200 Knoten) wäre zu erwarten, dass die kubische Schranke des Subgraph Algorithmus zunehmend dominiert, während VF2/RI auf solchen Graphen — je nach Dichte und Symmetriegrad — entweder weiterhin sub-exponentiell bleiben (geringe Dichte, viele eindeutige Labels) oder in den exponentiellen Worst Case geraten (hohe Symmetrie, viele Automorphismen). Eine systematische Messreihe für $n \in \{20, 30, 50, 100\}$ wäre daher ein wichtiger nächster Schritt.
- **Statistische Signifikanz:** Die hier verwendeten 8 bzw. 5 Wiederholungen pro Datenpunkt (Tabelle 2, Abbildung 9) genügen für eine erste Tendenzaussage, jedoch nicht für eine belastbare statistische Aussage (z. B. Konfidenzintervalle, t -Test auf Mittelwertunterschiede). Eine erweiterte Messreihe mit ≥ 100 Wiederholungen pro Konfiguration und expliziter Angabe von Konfidenzintervallen (95 %) wäre für eine Publikation in einer Fachzeitschrift erforderlich.
- **Implementierungssprache:** Die hier verwendete Referenzimplementierung des Subgraph Algorithmus ist reines Python mit `numpy`, während `networkx` ebenfalls in Python implementiert ist (mit C-Erweiterungen für einige Kernoperationen). Ein fairer Vergleich auf Implementierungsebene (z. B. beide Verfahren in C++ oder beide in einer JIT-kompilierten Umgebung wie `numba`) würde den Einfluss des Implementierungs-Overheads von der algorithmischen Komplexität trennen und die in Satz 1 bewiesene asymptotische Aussage von konstanten Faktoren isolieren.

7.13 Formale Charakterisierung des Übergangsbereichs θ

Die empirisch beobachtete Umkehrung des Geschwindigkeitsvorteils bei $n_B \approx 14$ (Abschnitt 6) wirft die Frage auf, ob sich der Schwellwert θ aus Abschnitt 7.11 formal als Funktion der Graphdichte p charakterisieren lässt. Intuitiv gilt: Je größer p , desto eher findet VF2 *sehr früh* einen Match oder einen Konsistenzbruch (da fast jeder Knoten mit fast jedem anderen verbunden ist, sind die meisten Teilabbildungen ϕ bereits nach wenigen Schritten konsistent oder inkonsistent), während die Kosten des Subgraph Algorithmus $\Theta(n^3)$ *unabhängig* von p sind (Satz 1 macht keine Annahme über p). Dies legt die folgende Vermutung nahe:

Bemerkung 6 (Vermutung: Dichteabhängigkeit von θ). Für $G(n, p)$ mit $p \rightarrow 1$ (vollständige Graphen) konvergiert die erwartete VF2-Laufzeit gegen $\Theta(n^2)$ (jede Teilabbildung ist sofort konsistent, das Pruning entscheidet nach $O(n)$ Rekursionsschritten mit $O(n)$ Aufwand pro Schritt für Konsistenzprüfung), während $T_{\text{Sub}}(n) = \Theta(n^3)$ unverändert bleibt. Für $p \rightarrow 0$ (sehr dünne Graphen) hingegen terminiert VF2 bei den meisten Knotenpaaren sofort durch Verletzung der Kantenbedingung, was ebenfalls zu $\Theta(n^2)$ oder besser führt. Der Schwellwert $\theta(p)$ wäre demnach für p nahe 0 und nahe 1 *kleiner* als für $p \approx 0,3$ – $0,5$ (Bereich maximaler struktureller Unsicherheit, in dem VF2 die meisten Rekursionsschritte benötigt) — d. h. $\theta(p)$ wäre als Funktion von p vermutlich *unimodal* mit Maximum bei mittlerer Dichte. Ein formaler Beweis dieser Vermutung würde eine probabilistische Analyse des VF2-Suchbaums auf $G(n, p)$ erfordern, analog zu den in [10] angedeuteten, aber nicht vollständig ausgeführten Average-Case-Betrachtungen.

Eine systematische Messreihe über das zweidimensionale Gitter $(n, p) \in \{4, \dots, 20\} \times \{0,1,0,2,\dots,0,9\}$ — eine direkte Erweiterung des in Abschnitt 6 durchgeführten Experiments — würde die in Bemerkung 6 formulierte Vermutung empirisch prüfbar machen und gleichzeitig eine Tabelle $\theta(p)$ liefern, mit der die Hybridstrategie aus Abschnitt 7.11 dichteadaptiv kalibriert werden könnte (d. h. θ würde nicht als globale Konstante, sondern als Funktion der für den jeweiligen Kandidaten k aus $m_k/(n_k(n_k - 1))$ geschätzten Dichte $\hat{p}_k$ nachgeschlagen).

7.14 Priorisierung der nächsten Entwicklungsschritte

Die beschriebenen Erweiterungen sollten in einer Reihenfolge umgesetzt werden, die zunächst die fachliche Korrektheit und Aussagekraft absichert und erst danach die Laufzeit optimiert. An erster Stelle steht daher die Validierung der vollständigen Suchsemantik: Für jede unterstützte Trefferrichtung muss geklärt sein, welche notwendigen Bedingungen die SQL-Vorauswahl verwenden darf. Die in Abschnitt 4 bewiesene Vollständigkeit gilt für $A_Q \preceq A_k$ bei vollständiger Kandidatenbetrachtung; ein endliches Kandidatenlimit kann Treffer abschneiden, und die umgekehrte Richtung $A_k \preceq A_Q$ erfordert eine entsprechend angepasste Vorauswahl. Diese Fälle sollten durch Tests mit gezielt konstruierten Netzwerken geprüft und in API sowie Benutzeroberfläche einheitlich kenntlich gemacht werden.

Darauf sollte eine reproduzierbare Evaluation mit realen biologischen Netzwerken und einer größeren Zahl statistisch auswertbarer Wiederholungen folgen. Sie würde zeigen, ob die auf synthetischen Graphen beobachteten Laufzeitverhältnisse und der Übergangsbereich θ auf unterschiedlichen Netzwerktypen und Dichten bestehen bleiben. Erst auf dieser belastbaren Grundlage lässt sich entscheiden, welche Optimierung den größten Nutzen verspricht: ein mehrdimensionaler Index für die Vorauswahl, eine adaptive Auswahl des Vergleichsverfahrens oder die Parallelisierung unabhängiger Kandidatenvergleiche. Erweiterungen um Labels und Kantengewichte sollten anschließend anhand biologischer Anwendungsfälle spezifiziert werden, damit eine verfeinerte Treffermenge nicht nur technisch berechenbar, sondern fachlich sinnvoll ist.

Diese Reihenfolge trennt drei Ziele, die bei der Weiterentwicklung nicht miteinander verwechselt werden sollten: Vollständigkeit und semantische Korrektheit der Suche, empirisch belastbare Aussagen über ihre Leistung sowie Optimierungen für konkrete Antwortzeit- und Skalierungsanforderungen. Insbesondere bleibt bei einer Hybridstrategie mit VF2 die Worst-Case-Garantie nur dann erhalten, wenn ein Verfahren mit garantierter Schranke — etwa der Subgraph Algorithmus — als Fallback vorgesehen ist.

7.15 Zusammenfassung

Diese Arbeit beschreibt Gen-DB als Full-Stack-System zur Suche in biologischen Netzwerken und verbindet die Architektur — einschließlich Paket- und Klassendiagrammen — mit der formalen Betrachtung der Such-Pipeline. Ihr zentraler Gedanke ist eine kostengünstige Vorauswahl in der Datenbank: Für die Richtung, in der das Query-Netzwerk Teil eines gespeicherten Netzwerks ist, werden Einträge mit zu wenigen Knoten oder Kanten vor dem strukturellen Vergleich ausgeschlossen. Diese notwendigen Bedingungen verkleinern den Suchraum, ohne bei vollständiger Kandidatenbetrachtung echte Treffer dieser Richtung auszuschließen; die endgültige Entscheidung trifft anschließend der Subgraph Algorithmus.

Für den strukturellen Vergleich wurde die Laufzeit $O(n^3)$ hergeleitet. In Verbindung mit dem relationalen Pre-Filter ergibt sich für C verbleibende Kandidaten eine Laufzeit von

$O(\log N + C) + O(C \cdot n^3)$. Bei fest begrenztem C und beschränkter Netzwerkgröße wächst der Indexsuchanteil weiterhin logarithmisch mit der Datenbankgröße; ein Kandidatenlimit begrenzt zwar die Zahl der Vergleiche, kann jedoch die Vollständigkeit beeinträchtigen. Die Beweise zur Vorauswahl gelten zudem ausdrücklich für $A_Q \preceq A_k$; eine Suche in der umgekehrten Richtung benötigt eigene Filterbedingungen.

Die Evaluation betrachtet eine synthetische Datenbank mit einer Million Netzwerken und vergleicht den Subgraph Algorithmus mit VF2. Die Messungen zeigen, dass kein einzelnes Verfahren in allen untersuchten Größenordnungen überlegen ist: VF2 kann durch sein Average-Case-Pruning bei größeren Instanzen schneller sein, während der Subgraph Algorithmus eine kubische Worst-Case-Schranke bietet. Die Resultate sind wegen der synthetischen Graphenfamilie und der begrenzten Zahl von Wiederholungen als erste empirische Einordnung zu verstehen, nicht als allgemeingültiger Laufzeitvergleich.

Aus diesen Ergebnissen folgt eine klare Entwicklungsrichtung: Zunächst sind Suchrichtungen, Filterbedingungen und Kandidatenlimits durch gezielte Tests abzusichern. Darauf sollten reproduzierbare Benchmarks mit realen biologischen Netzwerken folgen; erst dann lassen sich Indexierung, Hybridverfahren und Parallelisierung belastbar priorisieren. Erweiterungen um biologische Labels und Gewichte können die fachliche Aussagekraft der Suche erhöhen, müssen aber als präzisierte Treffermodellierung entworfen werden. Damit verbindet Gen-DB bereits eine klare modulare Architektur und eine nachvollziehbare Kandidatenreduktion mit einem strukturellen Vergleich; zugleich benennt die Arbeit die Korrektheits- und Evaluationsschritte, die für den Übergang von der prototypischen Realisierung zu einem belastbar validierten Suchsystem erforderlich sind.

Literatur

- [1] Stephan Epp. *Der Subgraph Algorithmus.*, 2026. Verfügbar unter: <https://github.com/naphets86/subgraph>.
- [2] Amir Abboud, Arturs Backurs, Virginia Vassilevska Williams. *Tight Hardness Results for LCS and Other Sequence Similarity Measures*. In: Proceedings of FOCS 2015.
- [3] Paul Erdős, Alfréd Rényi. *On Random Graphs I*. Publicationes Mathematicae (Debrecen), 6:290–297, 1959.
- [4] The PostgreSQL Global Development Group. *PostgreSQL 15 Documentation*. <https://www.postgresql.org/docs/15/>.
- [5] Sebastián Ramírez et al. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>.
- [6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*, 4. Auflage. MIT Press, 2022.
- [7] Richard P. Brent. *The Parallel Evaluation of General Arithmetic Expressions*. Journal of the ACM, 21(2):201–206, 1974.
- [8] Antonin Guttman. *R-Trees: A Dynamic Index Structure for Spatial Searching*. In: Proceedings of ACM SIGMOD, 1984.
- [9] Robert A. Wagner, Michael J. Fischer. *The String-to-String Correction Problem*. Journal of the ACM, 21(1):168–173, 1974.

- [10] Luigi P. Cordella, Pasquale Foggia, Carlo Sansone, Mario Vento. *A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 26(10):1367–1372, 2004.
- [11] Julian R. Ullmann. *An Algorithm for Subgraph Isomorphism*. Journal of the ACM, 23(1):31–42, 1976.
- [12] Aric A. Hagberg, Daniel A. Schult, Pieter J. Swart. *Exploring Network Structure, Dynamics, and Function using NetworkX*. In: Proceedings of the 7th Python in Science Conference (SciPy), 2008.
- [13] Vincenzo Bonnici, Rosalba Giugno, Alfredo Pulvirenti, Dennis Shasha, Alfredo Ferro. *A Subgraph Isomorphism Algorithm and Its Application to Biochemical Data*. BMC Bioinformatics, 14(Suppl 7):S13, 2013.
- [14] Huahai He, Ambuj K. Singh. *Graphs-at-a-time: Query Language and Access Methods for Graph Databases*. In: Proceedings of ACM SIGMOD, 2008.